

WALCHAND COLLEGE OF ENGINEERING, SANGLI

INDEX

Sr. No.	Expt. No.	Title of Experiment	Date on which		Page No.	Remarks
			Experiment Performed	Report Submitted		
	1	Study the use of graphs/charts	23/7			
	2	Perform Normalization	30/7			
	3	Perform Binning of data	6/8			
	4	Find Info Gain	13/8			
	5	Find t and d weight	20/8			
	6	Find 5 number summary	27/8			
	7	Find Frequent itemset	10/9			
	8	Extend Program 7, to Find association Rule.	17/9			
	9	Find correlation betn items/entities.	24/09			
	10	Distance and cluster	1/10			
	11	Write program for Agglomerative hierarchical clustering.	8/10			
	12	WAP to find Gain/Gini	15/10			
	13	WAP for Baye's classification	22/10			
	14	WAP for linear regression	22/10			
	15	WAP to implement any DM concept on complex data.	22/10			

Certified that Shri. _____

Roll No. 22620004 of _____ Class has completed satisfactorily the _____ experiment in _____ during the

Year 2024 - 20 25

Date :

IN CHARGE

HEAD OF THE
DEPARTMENT



Assignment : 2

Aim : To perform normalization of data using Min-Max and z-score normalization.

Theory : Normalization is a technique used to scale numerical data into specific range. The need for normalization arises because attributes in a data set may have different units or scales, which can lead to biased results. By normalizing the data, we ensure that all features contribute equally to the analysis.

► Types of normalization :

- 1] Min-Max Normalization
- 2] z-Score Normalization
- 3] Divide by n.

1] Min-Max Normalization:

- This technique scales data to fixed range, usually $[0, 1]$
- It linearly transforms data from original range to the desired range while maintaining the relation between data points.
- Formula:

$$V' = \frac{(V - \min(A))}{(\max(A) - \min(A))} \times (\text{new max}(A) - \text{new min}(A)) + \text{new min}(A)$$

- V' = Normalized Value
- V = Original Value
- $\min(A)$ & $\max(A)$ = minimum & maximum values
- $\left. \begin{array}{l} \text{new min}(A) \\ \text{new max}(A) \end{array} \right\}$ new desired range



2] Z-Score normalization :

- This technique transforms the data based on mean and standard deviation of the attribute.
- Useful when data is Gaussian or when outliers might skew the results of min-max normalization
- Formula :

$$v' = \frac{(v - \bar{A})}{\sigma_A}$$

v = Original Value

v' = normalized Value

$\bar{A}$ = mean

σ_A = Standard deviation

Numerical

1] Min-Max Normalization :

$$\text{new_min} = 0$$

$$\text{min} = 0.3065$$

$$\text{new_max} = 0.8$$

$$\text{max} = 1$$

	v	Formula	v_i
1]	0.33535	$\frac{(0.33535 - 0.3065) \times ((0.8 - 0) + 0)}{(1 - 0.3065)} \Rightarrow$	0.0
2]	0.33535	$\frac{(0.33535 - 0.3065) \times ((0.8 - 0) + 0)}{(1 - 0.3065)}$	0.0
3]	0.33003	$\frac{(0.33003 - 0.3065) \times ((0.8 - 0) + 0)}{(1 - 0.3065)}$	0.02
4]	0.97583	$\frac{(0.97583 - 0.3065) \times ((0.8 - 0) + 0)}{(1 - 0.3065)}$	0.77



Walchand College of Engineering, Sangli

5] 0.81571	$\frac{(0.81571 - 0.3065)}{(1 - 0.3065)} \times ((0.8 - 0) + 0)$	0.5874
6] 0.3065	$\frac{(0.3065 - 0.3065)}{(1 - 0.3065)} \times ((0.8 - 0) + 0)$	0
7] 0.36586	$\frac{(0.36586 - 0.3065)}{(1 - 0.3065)} \times ((0.8 - 0) + 0)$	0.0684
8] 0.32607	$\frac{(0.32607 - 0.3065)}{(1 - 0.3065)} \times ((0.8 - 0) + 0)$	0.0225
9] 1	$\frac{(1 - 0.3065)}{(1 - 0.3065)} \times ((0.8 - 0) + 0)$	0.8
10] 0.72507	$\frac{(0.72507 - 0.3065)}{(1 - 0.3065)} \times ((0.8 - 0) + 0)$	0.4828
11] 0.53474	$\frac{(0.53474 - 0.3065)}{(1 - 0.3065)} \times ((0.8 - 0) + 0)$	0.2632
12] 0.37471	$\frac{(0.37471 - 0.3065)}{(1 - 0.3065)} \times ((0.8 - 0) + 0)$	0.0786
13] 0.35607	$\frac{(0.35607 - 0.3065)}{(1 - 0.3065)} \times ((0.8 - 0) + 0)$	0.0571
14] 0.39577	$\frac{(0.39577 - 0.3065)}{(1 - 0.3065)} \times ((0.8 - 0) + 0)$	0.1029



2] z-score Normalization :

$$\rightarrow \text{Mean } (\bar{A}) = \frac{0.33535 + 0.33535 + 0.33003 + 0.97583 + 0.81571 + 0.3065 + 0.36586 + 0.32607 + 1 + 0.72507 + 0.53474 + 0.37471 + 0.35607 + 0.39577}{14}$$

$$\bar{A} = 0.5126$$

$$\bar{A} = \frac{\sum V}{n}$$

→ Standard deviation :

$$\sigma_A = \sqrt{\frac{\sum (V - \bar{A})^2}{n}}$$

	V	$(V - \bar{A})^2$
1]	0.33535	0.0314
2]	0.33535	0.0314
3]	0.33003	0.0333
4]	0.97583	0.2145
5]	0.81571	0.0918
6]	0.3065	0.0425
7]	0.36586	0.0215
8]	0.32607	0.0348
9]	1	0.2375
10]	0.72507	0.0451
11]	0.53474	0.0004
12]	0.37471	0.0190
13]	0.35607	0.0245
14]	0.39577	0.0136

$$\sigma_A = \sqrt{\frac{0.8413}{14}} = 0.24$$



	v (Original Value)	Formula	v' (Normalized Value)
1]	0.33535	$\frac{(0.33535 - 0.5126)}{0.2451}$	- 0.7232
2]	0.33535	$\frac{(0.33535 - 0.5126)}{0.2451}$	- 0.7232
3]	0.33003	$\frac{(0.33003 - 0.5126)}{0.2451}$	- 0.7449
4]	0.97583	$\frac{(0.97583 - 0.5126)}{0.2451}$	1.889
5]	0.81571	$\frac{(0.81571 - 0.5126)}{0.2451}$	1.2366
6]	0.3065	$\frac{(0.3065 - 0.5126)}{0.2451}$	- 0.842
7]	0.36586	$\frac{(0.36586 - 0.5126)}{0.2451}$	- 0.5986
8]	0.32607	$\frac{(0.32607 - 0.5126)}{0.2451}$	- 0.7610
9]	1	$\frac{(1 - 0.5126)}{0.2451}$	1.988
10]	0.72507	$\frac{(0.72507 - 0.5126)}{0.2451}$	0.8668
11]	0.53474	$\frac{(0.53474 - 0.5126)}{0.2451}$	0.0903
12]	0.37471	$\frac{(0.37471 - 0.5126)}{0.2451}$	- 0.562
13]	0.35607	$\frac{(0.35607 - 0.5126)}{0.2451}$	- 0.6386
14]	0.39577	$\frac{(0.39577 - 0.5126)}{0.2451}$	- 0.4766

Conclusion

Conclusion:

In conclusion, Min-max and z-score normalization techniques has helped me to understand how data can be scaled and standardized. It ensure that features are on a similar scale, improving the performance and accuracy.

~~Repeat~~

Conclusion:

Normalization is an important processing task in data pre-processing. It is used to ensure consistency in data records. In order to bring all attribute on the same scale min-max normalization, for z-score normalization is used, in order to standardize score on same scale by dividing a score deviation by standard deviation for that z-score normalization is useful.

VB

Assignment 3

Aim: To perform normalization of data using Min-Max and Z-score normalization.

Min-Max Normalization

1) Using program

Code:

```
import java.io.*;
import java.util.*;
import java.nio.file.*;

public class MinMaxNormalization {
    public static List<String> split(String line, char delimiter) {
        List<String> result = new ArrayList<>();
        String[] tokens = line.split(String.valueOf(delimiter));
        result.addAll(Arrays.asList(tokens));
        return result;
    }

    public static void main(String[] args) {
        double mini, maxi, newMini, newMaxi;
        List<Double> voltageData = new ArrayList<>();

        try (BufferedReader in = Files.newBufferedReader(Paths.get("Mine_Dataset.csv")));
            BufferedWriter out =
Files.newBufferedWriter(Paths.get("exp2_output_MinMax.csv"))) {

            String line = in.readLine();
            out.write(line);
            out.newLine();
```



```
while ((line = in.readLine()) != null) {  
    List<String> fields = split(line, ',');  
    double voltage = Double.parseDouble(fields.get(0));  
    voltageData.add(voltage);  
}  
  
mini = Collections.min(voltageData);  
maxi = Collections.max(voltageData);  
  
Scanner scanner = new Scanner(System.in);  
System.out.print("Enter new min: ");  
newMini = scanner.nextDouble();  
System.out.print("Enter new max: ");  
newMaxi = scanner.nextDouble();  
  
in.close();  
  
int index = 0;  
BufferedReader inAgain =  
Files.newBufferedReader(Paths.get("Mine_Dataset.csv"));  
inAgain.readLine();  
  
while ((line = inAgain.readLine()) != null) {  
    List<String> fields = split(line, ',');  
  
    double voltage = voltageData.get(index++);  
    double normalizedVoltage = ((voltage - mini) / (maxi - mini)) * (newMaxi -  
newMini) + newMini;  
  
    out.write(normalizedVoltage + "," + fields.get(1) + "," + fields.get(2) + "," +  
fields.get(3));  
    out.newLine();  
}
```

}

```
System.out.println("Min-Max Normalization applied successfully!");
```

```
} catch (Exception e) {
```

```
System.out.println("Error opening input file");
```

```
}
```

```
}
```

```
}
```

```
PS C:\Users\khushi\Desktop> javac MinMaxNormalization.java
PS C:\Users\khushi\Desktop> java MinMaxNormalization
Enter new min: 0
Enter new max: 0.8
Min-Max Normalization applied successfully and saved to exp2_output_MinMax.csv
PS C:\Users\khushi\Desktop>
```

Input (Mine_DataSet.csv):

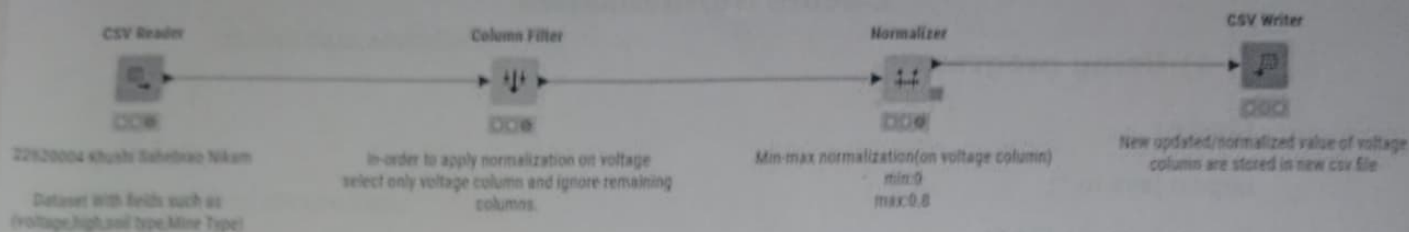
Voltage ▾	High ▾	Soil Ty ▾	Mine Ty ▾
0.33535	0.18182	0.6	1
0.33535	0.27273	0.6	1
0.33003	0.45455	0.6	1
0.97583	0.27273	0.6	2
0.81571	0.36364	0.6	2
0.3065	1	1	3
0.36586	0.09091	0.6	1
0.32607	0.36364	0.6	1
1	0.18182	0.6	2
0.72507	0.45455	0.6	2
0.53474	0.72727	0.6	2
0.37471	0.54545	0.6	3
0.35607	0.81818	0.6	3
0.39577	0.27273	0.2	3

Output (exp2_output_MinMax.csv):

Name: Nikam Khushi Sahebrao Nikam[22620004]

Voltage	High	Soil Type	Mine Type
0.033283	0.181818	0.6	1
0.033283	0.272727	0.6	1
0.027149	0.454545	0.6	1
0.772119	0.272727	0.6	2
0.58741	0.363636	0.6	2
0	1	1	3
0.068482	0.090909	0.6	1
0.022583	0.363636	0.6	1
0.8	0.181818	0.6	2
0.482858	0.454545	0.6	2
0.263298	0.727273	0.6	2
0.078686	0.545455	0.6	3
0.05719	0.818182	0.6	3
0.102984	0.272727	0.2	3

2) Using KNIME:



Input (Mine_DataSet.csv)

Voltage	High	Soil Ty	Mine Ty
0.33535	0.18182	0.6	1
0.33535	0.27273	0.6	1
0.33003	0.45455	0.6	1
0.97583	0.27273	0.6	2
0.81571	0.36364	0.6	2
0.3065	1	1	3
0.36586	0.09091	0.6	1
0.32607	0.36364	0.6	1
1	0.18182	0.6	2
0.72507	0.45455	0.6	2
0.53474	0.72727	0.6	2
0.37471	0.54545	0.6	3
0.35607	0.81818	0.6	3
0.39577	0.27273	0.2	3

Output (Mine_DataSet_minmax.csv)

Voltage
0.033283
0.033283
0.027149
0.772119
0.58741
0
0.068482
0.022583
0.8
0.482858
0.263298
0.078686
0.05719
0.102984

Z-Score Normalization

1) Using program

Code:

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class ZScoreNormalization {

    public static List<String> split(String line, char delimiter) {
        List<String> result = new ArrayList<>();
        String[] tokens = line.split(String.valueOf(delimiter));
        result.addAll(Arrays.asList(tokens));
        return result;
    }

    public static List<Double> zScoreNormalization(List<Double> data) {
        double sum = 0, mean, stddev = 0;

        for (double val : data) {
            sum += val;
        }

        mean = sum / data.size();

        for (double val : data) {
            stddev += Math.pow(val - mean, 2);
        }

        stddev = Math.sqrt(stddev / data.size());

        List<Double> zScoreData = new ArrayList<>();
```

```
for (double val : data) {
    double zScoreVal = (val - mean) / stddev;
    zScoreData.add(zScoreVal);
}

return zScoreData;
}

public static void main(String[] args) {
    List<Double> voltageData = new ArrayList<>();

    try (BufferedReader in = Files.newBufferedReader(Paths.get("Mine_Dataset.csv")));
        BufferedWriter out =
Files.newBufferedWriter(Paths.get("exp2_output_ZScore.csv"))) {

        String line = in.readLine();
        out.write(line);
        out.newLine();

        while ((line = in.readLine()) != null) {
            List<String> fields = split(line, ',');
            double voltage = Double.parseDouble(fields.get(0));
            voltageData.add(voltage);
        }

        List<Double> zScoreNormalized = zScoreNormalization(voltageData);

        in.close();

        BufferedReader inAgain =
Files.newBufferedReader(Paths.get("Mine_Dataset.csv"));
        inAgain.readLine();
    }
}
```



```
int index = 0;
while ((line = inAgain.readLine()) != null) {
    List<String> fields = split(line, ',');

    double normalizedVoltage = zScoreNormalized.get(index++);
    out.write(normalizedVoltage + "," + fields.get(1) + "," + fields.get(2) + "," +
fields.get(3));
    out.newLine();
}

System.out.println("Z-Score Normalization applied successfully");

} catch (Exception e) {
    System.out.println("Error!");
}
}
}
```

```
PS C:\Users\khushi\Desktop> javac ZScoreNormalization.java
PS C:\Users\khushi\Desktop> java ZScoreNormalization
Z-Score Normalization applied successfully and saved to exp2_output_ZScore.csv
```

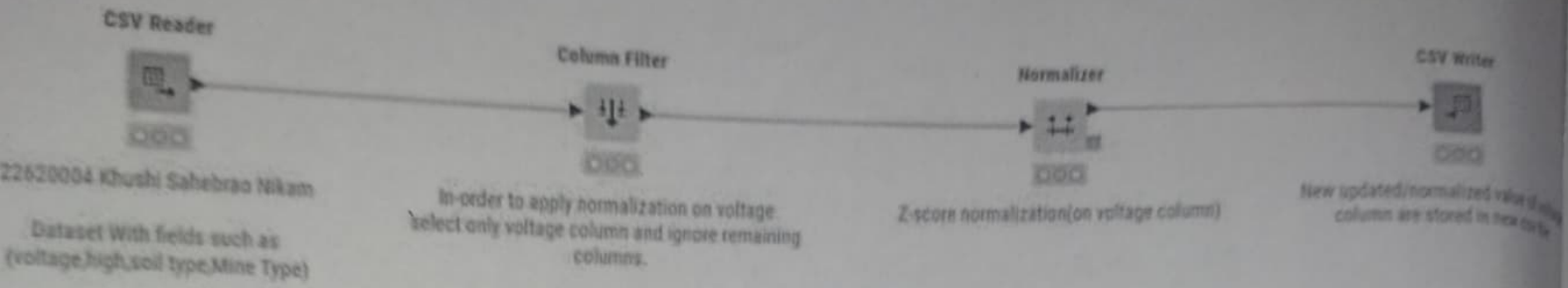
Input (Mine_DataSet.csv):

Voltage ▾	High ▾	Soil Ty ▾	Mine Ty ▾
0.33535	0.18182	0.6	1
0.33535	0.27273	0.6	1
0.33003	0.45455	0.6	1
0.97583	0.27273	0.6	2
0.81571	0.36364	0.6	2
0.3065	1	1	3
0.36586	0.09091	0.6	1
0.32607	0.36364	0.6	1
1	0.18182	0.6	2
0.72507	0.45455	0.6	2
0.53474	0.72727	0.6	2
0.37471	0.54545	0.6	3
0.35607	0.81818	0.6	3
0.39577	0.27273	0.2	3

Output (exp2_output_ZScore.csv)

Voltage	High	Soil Type	Mine Type
-0.72305	0.181818	0.6	1
-0.72305	0.272727	0.6	1
-0.74474	0.454545	0.6	1
1.888926	0.272727	0.6	2
1.235931	0.363636	0.6	2
-0.84072	1	1	3
-0.59861	0.090909	0.6	1
-0.76088	0.363636	0.6	1
1.987492	0.181818	0.6	2
0.866311	0.454545	0.6	2
0.09011	0.727273	0.6	2
-0.56254	0.545455	0.6	3
-0.63853	0.818182	0.6	3
-0.47664	0.272727	0.2	3

2) Using KNIME:



Input (Mine_DataSet.csv)

Voltage	High	Soil Ty	Mine Ty
0.33535	0.18182	0.6	1
0.33535	0.27273	0.6	1
0.33003	0.45455	0.6	1
0.97583	0.27273	0.6	2
0.81571	0.36364	0.6	2
0.3065	1	1	3
0.36586	0.09091	0.6	1
0.32607	0.36364	0.6	1
1	0.18182	0.6	2
0.72507	0.45455	0.6	2
0.53474	0.72727	0.6	2
0.37471	0.54545	0.6	3
0.35607	0.81818	0.6	3
0.39577	0.27273	0.2	3

Output (Mine_DataSet_zscore.csv)

Voltage
-0.69675
-0.69675
-0.71765
1.820215
1.190973
-0.81013
-0.57684
-0.7332
1.915195
0.834799
0.086832
-0.54208
-0.61531
-0.4593



Assignment 3

Aim: Perform Binning of data.

Theory:

Introduction to binning: Binning is a fundamental data processing technique used in data mining, data analysis. It is applied to smooth the data, reduce the effect of noise, and reveal meaningful patterns in datasets. Primary goal is to convert numerical values into a more understandable or simplified format by grouping them into intervals known as bins.

Types of binning / Methods of binning:

1] Smoothing by bins mean:

Each value in the bin is replaced by the mean value of the bin.

$$\mu_B = \frac{x_1 + x_2 + \dots + x_n}{n}$$

2] Smoothing by bin median:

Each value in bin is replaced by the median value of the bin.

if n is odd the M_B is $x_{(\frac{n+1}{2})}$

if n is even then M_B is average of two middle values.

3] Smoothing by boundaries:

Each value is replaced by the closet boundary value (either the minimum or the maximum value in the bin)



$$\text{new } x = \begin{cases} x_{\min} & \text{if } x \text{ is closer to } x_{\min} \\ x_{\max} & \text{if } x \text{ is closer to } x_{\max} \end{cases}$$

- Numerical

i) Given data in sorted form :

290000, 300000, 310000, 325000, 340000, 350000,
380000, 400000, 450000

ii) Bin size = 3

[290000, 300000, 310000]

[325000, 340000, 350000]

[380000, 400000, 450000]

iii) * smoothing by bin mean

$$\text{Bin 1 mean} = (290000 + 300000 + 310000) / 3 = 300000$$

$$\text{Replaced values} = [300000, 300000, 300000]$$

$$\text{Bin 2 mean} = (325000 + 340000 + 350000) / 3 = 338333$$

$$\text{Replaced values} = [338333, 338333, 338333]$$

$$\text{Bin 3 mean} = (380000 + 400000 + 450000) / 3 = 410000$$

$$\text{Replaced values} = [410000, 410000, 410000]$$

* smoothing by bin median :

$$\text{Bin 1 median} = 300000$$

$$\text{Replaced values} = [300000, 300000, 300000]$$

$$\text{Bin 2 median} = 340000$$

$$\text{Replaced values} = [340000, 340000, 340000]$$

$$\text{Bin 3 median} = 400000$$

$$\text{Replaced values} = [400000, 400000, 400000]$$



* Smoothing by bin boundaries:

Bin 1 boundary values : [290000, 310000]

Replaced values : [290000, 290000, 310000]

Bin 2 boundary values = [325000, 350000]

Replaced values : [325000, 325000, 350000]

Bin 3 boundary values = [380000, 450000]

Replaced values : [380000, 380000, 450000]

Conclusion :

Binning is a useful technique for smoothing data, minimizing noise and data to revealing patterns. Smoothing by bin mean offers a balanced coverage representation, while smoothing by median retains the central value of each bin. On the other hand, bin boundaries preserve the extreme values, maintaining the range of the data. Each method provides distinct advantages based on the specific goals of data analysis.

2/3

Assignment 3

Aim: Perform Binning of data.

1) Using program

Code:

```
import java.io.*;
import java.util.*;

public class DataBinning {

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String csvFile = "House_Prices_Dataset.csv";
        int columnIndex = 3;
        System.out.print("Enter the number of bins you want to create: ");
        int numBins = scanner.nextInt();

        System.out.println("\nChoose the type of binning you want to perform:");
        System.out.println("1. Equal Frequency Binning");
        System.out.println("2. Equal Width Binning");
        System.out.print("Your choice (1 or 2): ");
        int binningChoice = scanner.nextInt();
        System.out.println();

        List<Double> dataValues = readDataFromCsv(csvFile, columnIndex);
        if (dataValues.isEmpty()) {
            System.out.println("No data available for binning.");
            return;
        }
    }
}
```

```

Collections.sort(dataValues);
List<List<Double>> bins;

if (binningChoice == 1) {
    bins = equalFrequencyBinning(dataValues, numBins);
} else if (binningChoice == 2) {
    bins = equalWidthBinning(dataValues, numBins);
} else {
    System.out.println("Invalid choice. Exiting.");
    return;
}

System.out.println("Binning results:");
for (int binCount = 0; binCount < bins.size(); binCount++) {
    List<Double> bin = bins.get(binCount);
    System.out.println("Bin " + (binCount + 1) + " contents: " + bin);
    System.out.printf("Smoothing by Bin Mean: %.2f%n", smoothByMean(bin));
    System.out.printf("Smoothing by Bin Median: %.2f%n", smoothByMedian(bin));
    System.out.printf("Smoothing by Boundaries: %s%n", smoothByBoundaries(bin));
    System.out.println("-----");
}

scanner.close();
}

private static List<Double> readDataFromCsv(String csvFile, int columnIndex) {
    List<Double> dataValues = new ArrayList<>();
    try (BufferedReader br = new BufferedReader(new FileReader(csvFile))) {
        br.readLine();
        String line;
        while ((line = br.readLine()) != null) {
            String[] values = line.split(",");

```

```

        if (values.length > columnIndex) {
            try {
                dataValues.add(Double.parseDouble(values[columnIndex]));
            } catch (NumberFormatException e) {
                System.err.println("Invalid number format in line: " + line);
            }
        }
    }
} catch (IOException e) {
    System.err.println("Error reading file: " + e.getMessage());
}
return dataValues;
}

```

```

private static List<List<Double>> equalFrequencyBinning(List<Double> dataValues,
int numBins) {

```

```

    List<List<Double>> bins = new ArrayList<>();
    int binSize = dataValues.size() / numBins;
    for (int i = 0; i < dataValues.size(); i += binSize) {
        int end = Math.min(i + binSize, dataValues.size());
        bins.add(new ArrayList<>(dataValues.subList(i, end)));
    }
    return bins;
}

```

```

private static List<List<Double>> equalWidthBinning(List<Double> dataValues, int
numBins) {

```

```

    List<List<Double>> bins = new ArrayList<>();
    double min = Collections.min(dataValues);
    double max = Collections.max(dataValues);
    double binWidth = (max - min) / numBins;

```



```

for (int i = 0; i < numBins; i++) {
    double binLowerBound = min + i * binWidth;
    double binUpperBound = min + (i + 1) * binWidth;
    List<Double> bin = new ArrayList<>();
    for (Double value : dataValues) {
        if (value >= binLowerBound && value < binUpperBound) {
            bin.add(value);
        }
    }
    bins.add(bin);
}
bins.get(bins.size() - 1).add(max);
return bins;
}

private static double smoothByMean(List<Double> bin) {
    return bin.stream().mapToDouble(Double::doubleValue).average().orElse(0.0);
}

private static double smoothByMedian(List<Double> bin) {
    Collections.sort(bin);
    int size = bin.size();
    return size % 2 == 0
        ? (bin.get(size / 2 - 1) + bin.get(size / 2)) / 2.0
        : bin.get(size / 2);
}

private static List<Double> smoothByBoundaries(List<Double> bin) {
    double min = Collections.min(bin);
    double max = Collections.max(bin);
    List<Double> boundarySmoothBin = new ArrayList<>();
    for (Double value : bin) {

```

```
        boundarySmoothBin.add(Math.abs(value - min) < Math.abs(value - max) ? min :
max);
    }
    return boundarySmoothBin;
}
}
```

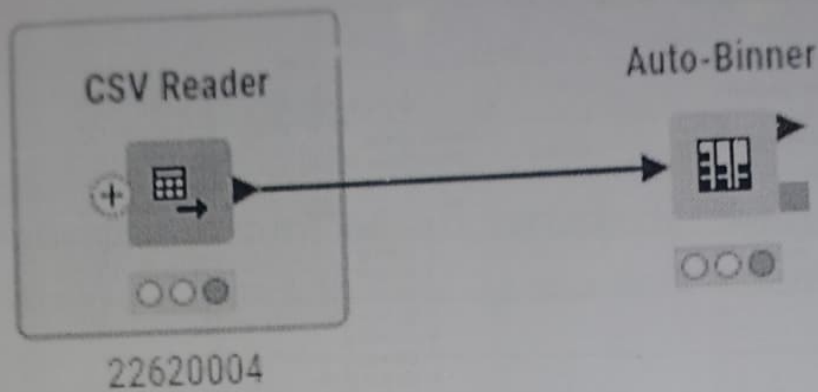
```
PS C:\Users\khushi\Desktop\DM\assignment3> java DataBinning
Enter the number of bins you want to create: 3

Choose the type of binning you want to perform:
1. Equal Frequency Binning
2. Equal Width Binning
Your choice (1 or 2): 1

Binning results:
Bin 1 contents: [290000.0, 295000.0, 300000.0, 300000.0, 305000.0, 310000.0, 310000.0, 315000.0, 320000.0, 325000.0]
Smoothing by Bin Mean: 307000.00
Smoothing by Bin Median: 307500.00
Smoothing by Boundaries: [290000.0, 290000.0, 290000.0, 290000.0, 290000.0, 325000.0, 325000.0, 325000.0, 325000.0, 325000.0]
-----
Bin 2 contents: [330000.0, 335000.0, 340000.0, 345000.0, 350000.0, 350000.0, 355000.0, 360000.0, 380000.0, 385000.0]
Smoothing by Bin Mean: 353000.00
Smoothing by Bin Median: 350000.00
Smoothing by Boundaries: [330000.0, 330000.0, 330000.0, 330000.0, 330000.0, 330000.0, 330000.0, 385000.0, 385000.0, 385000.0]
-----
Bin 3 contents: [390000.0, 400000.0, 405000.0, 410000.0, 420000.0, 425000.0, 430000.0, 450000.0, 455000.0, 460000.0]
Smoothing by Bin Mean: 424500.00
Smoothing by Bin Median: 422500.00
Smoothing by Boundaries: [390000.0, 390000.0, 390000.0, 390000.0, 390000.0, 460000.0, 460000.0, 460000.0, 460000.0, 460000.0]
-----
```

	A	B	C	D
1	House_ID	Location	Size (sq ft)	Price (\$)
2	1	City A	1500	300000
3	2	City B	2000	350000
4	3	City A	1800	325000
5	4	City C	2200	400000
6	5	City B	1700	310000
7	6	City A	1600	290000
8	7	City C	2500	450000
9	8	City B	1900	340000
10	9	City A	2100	380000
11	10	City C	2400	420000
12	11	City A	1550	305000
13	12	City B	2050	355000
14	13	City A	1850	330000
15	14	City C	2250	405000
16	15	City B	1750	315000
17	16	City A	1650	295000
18	17	City C	2550	455000
19	18	City B	1950	345000
20	19	City A	2150	385000
21	20	City C	2450	425000
22	21	City A	1600	310000
23	22	City B	2100	360000

Input (House_Prices_Dataset.csv):



2) Using KNIME:

Output:

Rows: 30 | Columns: 5

#	RowID	House_ID Number (integer)	Location String	Size (sq ft) Number (integer)	Price (\$) Number (integer)	Price (\$) [Binned] String
1	Row0	1	City A	1500	300000	298,500
2	Row1	2	City B	2000	350000	349,500
3	Row2	3	City A	1800	325000	332,500
4	Row3	4	City C	2200	400000	400,500
5	Row4	5	City B	1700	310000	315,500
6	Row5	6	City A	1600	290000	298,500
7	Row6	7	City C	2500	450000	451,500
8	Row7	8	City B	1900	340000	332,500
9	Row8	9	City A	2100	380000	383,500
10	Row9	10	City C	2400	420000	417,500
11	Row...	11	City A	1550	305000	298,500
12	Row...	12	City B	2050	355000	349,500
13	Row...	13	City A	1850	330000	332,500
14	Row...	14	City C	2250	405000	400,500
15	Row...	15	City B	1750	315000	315,500
16	Row...	16	City A	1650	295000	298,500
17	Row...	17	City C	2550	455000	451,500
18	Row...	18	City B	1950	345000	332,500
19	Row...	19	City A	2150	385000	383,500
20	Row...	20	City C	2450	425000	417,500
21	Row...	21	City A	1600	310000	315,500
22	Row...	22	City B	2100	360000	366,500
23	Row...	23	City A	1900	335000	332,500

Weka:

weka explorer

Preprocess Classify Cluster Associate Select attributes Visualize

Open file... Open URL... Open DB... Generate... Undo Edit... Save...

Filter Choose Discretize -Y -B 10 -M -1.0 -R first-last -precision 4 Apply Stop

Current relation
Relation: House_Prices_Dataset-weka.filters.unsupervised.attribute....
Instances: 30 Attributes: 4 Sum of weights: 30

Attributes
All None Invert Pattern

No.	Name
1	House_ID
2	Location
3	Size (sq ft)
4	Price (\$)

Remove

Status OK

Selected attribute
Name: House_ID
Missing: 0 (0%)
Distinct: 10
Type: Nominal
Unique: 0 (0%)

No.	Label	Count	Weight
1	'(-inf-3.9]'	3	3
2	'(3.9-6.8]'	3	3
3	'(6.8-9.7]'	3	3
4	'(9.7-12.6]'	3	3
5	'(12.6-15.5]'	3	3
6	'(15.5-18.4]'	3	3
7	'(18.4-21.3]'	3	3
8	'(21.3-24.2]'	3	3
9	'(24.2-27.1]'	3	3

Class: Price (\$) (Nom)

Visualize All

Log x0



Assignment 4

Aim: Find Info Gain of an attribute from given data.

Theory:

Information gain is a measure used in decision trees to determine which attribute need to split on at each step. It quantifies difference in entropy (a measure of uncertainty or disorder) before and after a dataset is split by an attribute. The attribute with the highest information gain is selected as the best split point.

steps:

- 1] Entropy of the dataset (D): Entropy $H(D)$ measures the disorder or randomness in the data. For binary classification, entropy is defined as:

$$H(D) = -p_1 \log_2(p_1) - p_2 \log_2(p_2)$$

where p_1 & p_2 are the proportions of the two classes (Yes and No).

- 2] Entropy after splitting by an attribute: The entropy after a split on an attribute A is computed as

$$H(D/A) = \sum \left(\frac{|D_i|}{|D|} \right) H(D_i)$$

D_i is subset of data for each value of attribute.

- 3] Information Gain (IG): The information gain for splitting on attribute A is the difference between the original entropy and the entropy after splitting,

$$IG(A) = H(D) - H(D/A)$$



Numerical : Yes (9) and No (5) Total (14)

Outlook	Play	NoPlay	
Sunny	2	3 ✓	(5)
Overcast	4	0	(4)
Rain	3	2	(5)
	(9)	(5)	

Temperature	Play	NoPlay	
Hot	2	2	(4)
Cool	3	1	(4)
Mild	4	2	(6)
	(9)	(5)	

Humidity	Play	NoPlay	
High	3 ✓	4 ✓	(7)
Normal	6	1	(7)
	(9)	(5)	

Wind	Play	NoPlay	
True	3	3	(6)
False	6	2	(8)
	(9)	(5)	

Parent Entropy calculation:

Play = 9 NoPlay = 5

$$\text{Parent Entropy} = - \left(\frac{9}{14} \log_2 \frac{9}{14} + \frac{5}{14} \log_2 \frac{5}{14} \right)$$

$$= \underline{\underline{0.94}}$$

i) for outlook :

A) Entropy(Sunny) = $- \left(\frac{2}{5} \log_2 \frac{2}{5} + \frac{3}{5} \log_2 \frac{3}{5} \right)$

$$= 0.971$$



$$B) \text{ Entropy (Overcast)} = - \left(\frac{4}{4} \log_2 \frac{4}{4} + \frac{0}{4} \log_2 \frac{0}{4} \right) = 0$$

$$C) \text{ Entropy (Rain)} = - \left(\frac{3}{5} \log_2 \frac{3}{5} + \frac{2}{5} \log_2 \frac{2}{5} \right) = 0.971$$

$$\text{Total Entropy of outlook} = 0.693$$

$$\text{Info Gain} = 0.940 - \left[\left(\frac{5}{14} \times 0.971 \right) + \left(\frac{4}{14} \times 0 \right) + \left(\frac{5}{14} \times 0.971 \right) \right]$$

$$= 0.940 - 0.693$$

$$= \underline{\underline{0.247}}$$

ii) For Temperature :

$$A) \text{ Entropy (Hot)} = - \left(\frac{2}{4} \log_2 \frac{2}{4} + \frac{2}{4} \log_2 \frac{2}{4} \right) = 1.00$$

$$B) \text{ Entropy (Mild)} = - \left(\frac{4}{6} \log_2 \frac{4}{6} + \frac{2}{6} \log_2 \frac{2}{6} \right) = 0.918$$

$$C) \text{ Entropy (Cool)} = - \left(\frac{3}{4} \log_2 \frac{3}{4} + \frac{1}{4} \log_2 \frac{1}{4} \right) = 0.811$$

$$\text{Total Entropy of Temperature} = 0.911$$

$$\text{Info Gain} = 0.940 - \left[\left(\frac{4}{14} \times 1.0 \right) + \left(\frac{6}{14} \times 0.918 \right) + \left(\frac{4}{14} \times 0.811 \right) \right]$$

$$= 0.940 - 0.911$$

$$= \underline{\underline{0.029}}$$

iii) For Humidity :

$$A) \text{ Entropy (High)} = - \left(\frac{3}{7} \log_2 \frac{3}{7} + \frac{4}{7} \log_2 \frac{4}{7} \right) = 0.985$$

$$B) \text{ Entropy (Normal)} = - \left(\frac{6}{7} \log_2 \frac{6}{7} + \frac{1}{7} \log_2 \frac{1}{7} \right) = 0.592$$

$$\text{Total Entropy for Humidity} = 0.789$$

$$\text{Info Gain} = (0.940 - 0.789)$$

$$= \underline{\underline{0.151}}$$



For wind =

$$A) \text{Entropy (Weak)} = - \left(\frac{6}{8} \log_2 \frac{6}{8} + \frac{2}{8} \log_2 \frac{2}{8} \right) = 0.811$$

$$B) \text{Entropy (Strong)} = - \left(\frac{3}{6} \log_2 \frac{3}{6} + \frac{3}{6} \log_2 \frac{3}{6} \right) = 1.000$$

$$\text{Total Entropy for wind} = \underline{0.892}$$

$$\text{Info Gain} = 0.940 - 0.892$$

$$= \underline{0.048}$$

Conclusion :

Information Gain (IG) is a crucial metric in decision tree learning that measures how effectively an attribute separates data by calculating the reduction in entropy. Higher IG values indicate better predictors, with values ranging from 0 (no information) to 1 (perfect separation). This is essential for feature selection in machine learning, helping build optimal decision trees by placing the most informative attributes closer to the root node, thus enabling efficient and accurate decision-making processes.

Assignment 4

Aim: Find Info Gain of an attribute from given data.

1) Using program

Code:

```
import java.io.*;
import java.util.*;

public class InfoGainCalculator {

    public static void main(String[] args) {
        String csvFile = "football_play_data.csv";
        String line;
        String csvSplitBy = ",";
        int total = 0;
        int yesCount = 0;
        Map<String, int[]> childCount = new HashMap<>();
        Scanner scanner = new Scanner(System.in);

        List<String> columnHeaders = new ArrayList<>();
        try (BufferedReader br = new BufferedReader(new FileReader(csvFile))) {
            String headerLine = br.readLine();
            if (headerLine != null) {
                columnHeaders = Arrays.asList(headerLine.split(csvSplitBy));
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
        return;
```

}

```
System.out.println("Available columns:");
```

```
for (int i = 0; i < columnHeader.size() - 1; i++) {
```

```
    System.out.println(i + ": " + columnHeader.get(i));
```

```
}
```

```
System.out.print("Enter the column number to calculate information gain: ");
```

```
int choice = scanner.nextInt();
```

```
scanner.close();
```

```
if (choice < 0 || choice >= columnHeader.size() - 1) {
```

```
    System.out.println("Invalid column choice. Exiting.");
```

```
    return;
```

```
}
```

```
try (BufferedReader br = new BufferedReader(new FileReader(csvFile))) {
```

```
    br.readLine();
```

```
    while ((line = br.readLine()) != null) {
```

```
        String[] values = line.split(csvSplitBy);
```

```
        total++;
```

```
        if (values[4].equals("Yes")) {
```

```
            yesCount++;
```

```
        }
```

```
        String childName = values[choice];
```

```
        childCount.putIfAbsent(childName, new int[2]);
```

```
        if (values[4].equals("Yes")) {
```

```
            childCount.get(childName)[0]++;
```

```
        } else {
```

```
            childCount.get(childName)[1]++;
```

```

    }
}
} catch (IOException e) {
    e.printStackTrace();
}

double parentEntropy = calculateEntropy(yesCount, total);
double childEntropy = 0;

System.out.println("\n=== Calculation Steps ===");
System.out.println("Parent Entropy: " + String.format("%.4f",
parentEntropy));

System.out.println("\n | Child Node | Yes Count | No Count | Total | Entropy
|");
System.out.println("-----");

for (Map.Entry<String, int[]> entry : childCount.entrySet()) {
    int[] counts = entry.getValue();
    int childTotal = counts[0] + counts[1];
    double childNodeEntropy = calculateEntropy(counts[0], childTotal);
    childEntropy += ((double) childTotal / total) * childNodeEntropy;

    System.out.printf(" | %-10s | %-9d | %-8d | %-5d | %-8.4f%n",
        entry.getKey(), counts[0], counts[1], childTotal, childNodeEntropy);
}

System.out.println("-----");
System.out.printf("Total Child Entropy: %.4f%n", childEntropy);

double infoGain = parentEntropy - childEntropy;
System.out.printf("Information Gain: %.4f%n", infoGain);

```


}

```

public static double calculateEntropy(int yesCount, int total) {
    if (total == 0) return 0;
    double pYes = (double) yesCount / total;
    double pNo = 1 - pYes;
    return -pYes * log2(pYes) - pNo * log2(pNo);
}

```

```

public static double log2(double value) {
    if (value == 0) return 0;
    return Math.log(value) / Math.log(2);
}

```

```

PS C:\Users\khushi\Desktop\DM\assignment4> javac .\InfoGainCalculator.java
PS C:\Users\khushi\Desktop\DM\assignment4> java InfoGainCalculator
Available columns:

```

```

0: Outlook
1: Temperature
2: Humidity
3: Wind

```

```

Enter the column number to calculate information gain: 2

```

```

=== Calculation Steps ===
Parent Entropy: 0.9403

```

Child Node	Yes Count	No Count	Total	Entropy
High	3	4	7	0.9852
Normal	6	1	7	0.5917

```

Total Child Entropy: 0.7885
Information Gain: 0.1518

```

```

PS C:\Users\khushi\Desktop\DM\assignment4>

```

Input (football_play_data.csv):

Outlook	Temperature	Humidity	Wind	Played Football
Sunny	Hot	High	Weak	No
Sunny	Hot	High	Strong	No
Overcast	Hot	High	Weak	Yes
Rain	Mild	High	Weak	Yes
Rain	Cool	Normal	Weak	Yes
Rain	Cool	Normal	Strong	No
Overcast	Cool	Normal	Strong	Yes
Sunny	Mild	High	Weak	No
Sunny	Cool	Normal	Weak	Yes
Rain	Mild	Normal	Weak	Yes
Sunny	Mild	Normal	Strong	Yes
Overcast	Mild	High	Strong	Yes
Overcast	Hot	Normal	Weak	Yes
Rain	Mild	High	Strong	No

2) Using KNIME:



Output:

Clustering statistics

Data Statistics

Statistics	Value
Number of clusters found:	2
Number of objects in clusters:	14
Number of reference clusters:	2
Total number of patterns:	14

Data Statistics

Score	Value
Entropy:	0.8482
Quality:	0.1518

Row ID	I Size	D Entropy	D Normali...	D Quality
No	5	0.722	0.722	?
Yes	9	0.918	0.918	?
Overall	14	0.848	0.848	0.152

Preprocess Classify Cluster Associate Select attributes Visualize

Attribute Evaluator

Choose InfoGainAttributeEval

Search Method

Choose Ranker -T -1.7976931348623157E308 -N -1

Attribute Selection Mode

☒ Use full training set☐ Cross-validation

Folds 10

Seed 1

No class

Start

Stop

Result list (right-click for options)

11:31:26 - Ranker + InfoGainAttributeEval

Attribute selection output

=== Run information ===

Evaluator: weka.attributeSelection.InfoGainAttributeEval
 Search: weka.attributeSelection.Ranker -T -1.7976931348623157E308 -N -1
 Relation: football_play_data
 Instances: 14
 Attributes: 5
 Outlook
 Temperature
 Humidity
 Wind
 Played Football

Evaluation mode: evaluate on all training data

=== Attribute Selection on all input data ===

Search Method:
 Attribute ranking.

Attribute Evaluator (supervised, Class (nominal): 5 Played Football):
 Information Gain Ranking Filter

Ranked attributes:
 0.2467 1 Outlook
 0.1518 3 Humidity
 0.0481 4 Wind
 0.0292 2 Temperature

Selected attributes: 1,3,4,2 : 4

Status

OK

Log

x 0



Assignment 5

Aim:

Find t and d weight of a data:

Theory:

The T-weight and D-weight allows us to analyze the distribution of products across different regions or stores.

* T-weight (%)

This is percentage of a product sold in a particular region/store relative to the total quantity of that product sold across all stores/regions.

* D-weight (%)

This is percentage of a product sold in particular region/store relative to total quantity of all sold product in that region/store

Algorithm:

- 1] Find the total count of each product.
- 2] Find T-weight
- 3] Find D-weight
- 4] Tabulate the result

Numerical :

Region	Product	Quantity
Africa	SmartPhone	300
South America	SmartPhone	250
Australia	SmartPhone	450
Africa	Laptop	1000
South America	Laptop	1200
Australia	Laptop	200



Walchand College Of Engineering, Sangli.

Region	Smart Phone (Count)	Smart Phone (T-weight %)	Smart Phone (D-weight %)	laptop (count)	laptop (T-weight %)	laptop (D-weight %)
Africa	300	30%	23.08%	1000	41.67%	76.92%
SA	250	25%	17.24%	1200	50%	82.76%
Australia	450	45%	69.23%	200	8.33%	30.77%
Total	1000	100%		2400	100%	

smartPhone	
Total (Count)	T-weights : $(300/1000) \times 100 = 30\%$ $(250/1000) \times 100 = 25\%$ $(450/1000) \times 100 = 45\%$
1300	D-weights : $(300/1300) \times 100 = 23.08\%$
1450	$(250/1450) \times 100 = 17.24\%$
650	$(450/650) \times 100 = 69.23\%$
3400	similarly for laptops.

Conclusion :

The conclusion of T-weight and D-weight offers insights into both the overall distribution of a product (T-weight) and its importance within individual regions or stores (D-weights). T-weight shows the share of a product across all locations, helping identify where it performs best. D-weight reveals the product's significance within each specific region, indicating local preferences. By combining these two metrics, business can make informed decisions about product distribution.

Assignment 5

Aim: Find t and d weight of a data.

1) Using program

Code:

```
import java.io.*;
import java.util.*;

public class Weights {
    public static void main(String[] args) {
        String inputFilePath = "region_product_quantity.csv";
        String outputFilePath = "output_region_product_quantity.csv";

        Map<String, Integer> regionSmartphoneMap = new LinkedHashMap<>();
        Map<String, Integer> regionLaptopMap = new LinkedHashMap<>();
        int totalSmartphones = 0;
        int totalLaptops = 0;
        Map<String, Integer> regionTotalMap = new LinkedHashMap<>();

        try (BufferedReader br = new BufferedReader(new
        FileReader(inputFilePath))) {
            String line;
            boolean isHeader = true;

            while ((line = br.readLine()) != null) {
                if (isHeader) {
                    isHeader = false;
                    continue;
                }
            }
        }
```

```

    }

    String[] fields = line.split(",");
    String region = fields[0].trim();
    String product = fields[1].trim();
    int quantity = Integer.parseInt(fields[2].trim());

    if (product.equalsIgnoreCase("Smartphone")) {
        regionSmartphoneMap.put(region,
            regionSmartphoneMap.getOrDefault(region, 0) + quantity);
        totalSmartphones += quantity;
    } else if (product.equalsIgnoreCase("Laptop")) {
        regionLaptopMap.put(region, regionLaptopMap.getOrDefault(region,
            0) + quantity);
        totalLaptops += quantity;
    }

    regionTotalMap.put(region, regionTotalMap.getOrDefault(region, 0) +
        quantity);
}

regionTotalMap.put("Total", totalSmartphones + totalLaptops);

} catch (IOException e) {
    e.printStackTrace();
}

try (BufferedWriter bw = new BufferedWriter(new
    FileWriter(outputFilePath))) {
    bw.write("Region,Smartphone (Count),Smartphone (TWeight
        %),Smartphone (DWeight %),"

```

```

+ "Laptop (Count),Laptop (TWeight %),Laptop (DWeight %),Total
(Count)");

```

```

bw.newLine();

```

```

for (String region : regionTotalMap.keySet()) {

```

```

    if (!region.equals("Total")) {

```

```

        int smartphoneCount = regionSmartphoneMap.getDefault(region,
0);

```

```

        int laptopCount = regionLaptopMap.getDefault(region, 0);

```

```

        int totalCount = regionTotalMap.get(region);

```

```

        double smartphoneTWeight = totalCount > 0 ? ((double)
smartphoneCount / totalCount) * 100 : 0;

```

```

        double laptopTWeight = totalCount > 0 ? ((double) laptopCount /
totalCount) * 100 : 0;

```

```

        double smartphoneDWeight = totalCount > 0 ? ((double)
smartphoneCount / totalCount) * 100 : 0;

```

```

        double laptopDWeight = totalCount > 0 ? ((double) laptopCount /
totalCount) * 100 : 0;

```

```

        bw.write(String.format("%s,%d,%.2f%%,%.2f%%,%d,%.2f%%,%.2f
%%,%d\n",

```

```

            region, smartphoneCount, smartphoneTWeight,
smartphoneDWeight,

```

```

            laptopCount, laptopTWeight, laptopDWeight, totalCount));

```

```

    }

```

```

}

```

```

bw.write(String.format("Total,%d,100%%,100%%,%d,100%%,100%%,%d
%n",

```

```

    totalSmartphones, totalLaptops, regionTotalMap.get("Total")));

```

```

} catch (IOException e) {

```



```

        e.printStackTrace();
    }

    System.out.printf("Processing complete! Output written to: %s%n",
        outputFilePath);
}
}

```

```

PS C:\Users\khushi\Desktop\DM\Assignment5> javac Weights.java
PS C:\Users\khushi\Desktop\DM\Assignment5> java Weights
Processing complete! Output written to: output_region_product_quantity.csv

```

Input (region_product_data.csv):

Region	Product	Quantity (in thousands)
Africa	Smartpho	300
South Am	Smartpho	250
Australia	Smartpho	450
Africa	Laptop	1000
South Am	Laptop	1200
Australia	Laptop	200

output (output_region_product_data.csv):

Region	Smartphone	Smartphone (DWeight %)	Smartphone (DWeight %)	Laptop (Count)	Laptop (TWeight %)	Laptop (DWeight %)	Total (Count)
Africa	300	30.00%	23.08%	1000	41.67%	76.92%	1300
South America	250	25.00%	17.24%	1200	50.00%	82.76%	1450
Australia	450	45.00%	69.23%	200	8.33%	30.77%	650
Total	1000	100%	100%	2400	100%	100%	3400



Walchand College of Engineering, Sangli

Assignment : 6

Aim: Finding 5 number summary for dataset and study of box plot.

Theory: The 5-Number summary is a set of descriptive statistics that provides information about the distribution of a dataset. Consist of

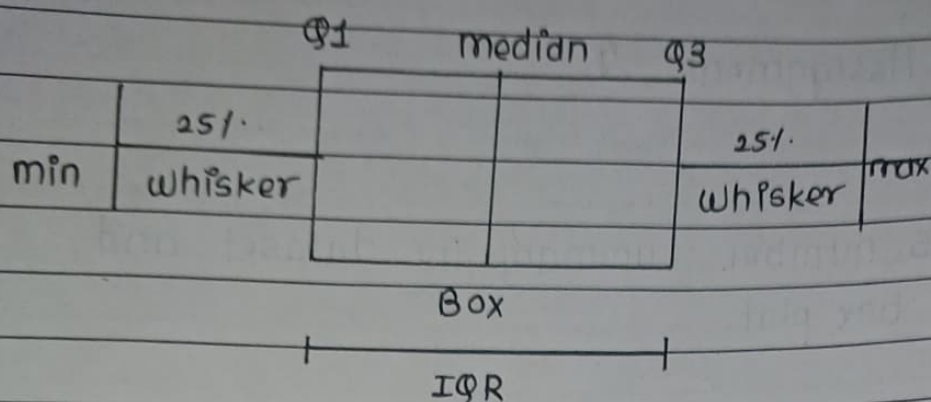
1. Minimum (smallest data point)
2. Maximum (largest data point)
3. Q_1 (First quartile)
4. Median (Middle value)
5. Q_3 (Third quartile)

Box-plot :

- Also known as box-and-whisker plot
- It is graphical representation of the 5 Number summary that visually shows the distribution of a dataset, including its spread and outliers, key elements :
 - 1] Box : Represents Interquartile Range (IQR), which is spread between Q_1 and Q_3 .
 - 2] Median line : A line inside box representing median (Q_2)
 - 3] Whisker : Line extending from the box to the smallest and largest value within 1.5 times the IQR.
 - 4] Outliers : Data points that lie outside the whiskers.



Walchand College of Engineering, Sangli



Numerical :

Voltage (V)

0.033283

0.033283

0.02714

0.7721

0.5874

0

0.0684

0.0225

0.8

0.4828

0.2632

0.0786

0.0571

0.1029

1] sort the data

→ 0.0, 0.0225, 0.02714, 0.033283, 0.033283, 0.0571, 0.0684, 0.0786, 0.1029, 0.2632, 0.4828, 0.5874, 0.7721, 0.8

2] Minimum = 0.0

Maximum = 0.8

3] Median (middle value)

$$\frac{0.0684 + 0.0786}{2}$$

$$= 0.0735$$

4] First Quartile Q_1 .

median of 0.0, 0.0225, 0.02714, 0.03328, 0.03328, 0.0571

$$= \frac{0.02714 + 0.03328}{2} = 0.030$$



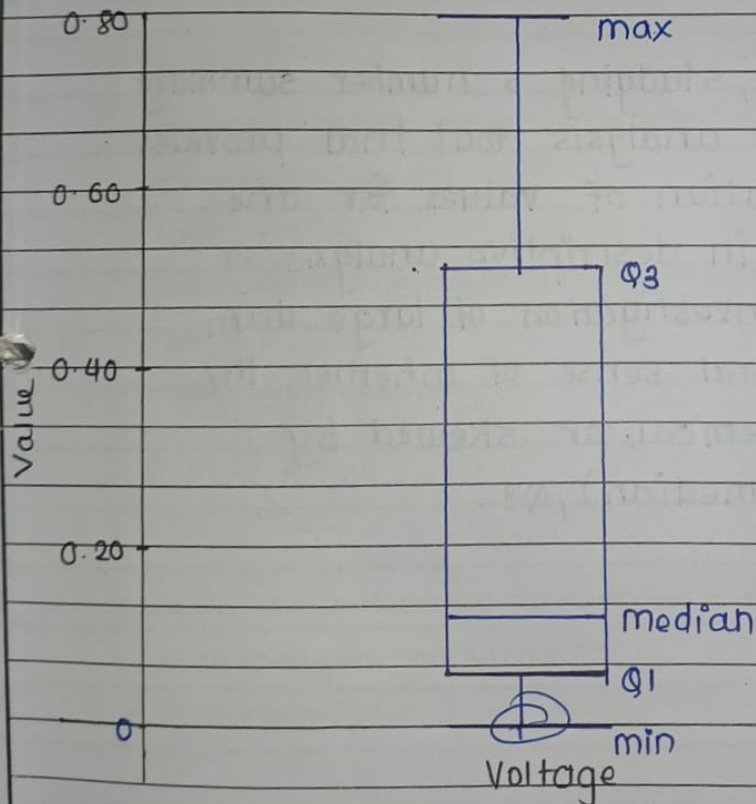
Walchand College of Engineering, Sangli

5) Third Quartile (Q_3)

median of 0.1029, 0.2632, 0.4828, 0.5874,
0.7721, 0.8

$$\frac{0.4828 + 0.5874}{2} = 0.535$$

Minimum	0.0
Q_1	0.0302
Median	0.0735
Q_3	0.535
Maximum	0.8



Box plot



Walchand College of Engineering, Sangli

Conclusion:

In conclusion, studying the Five-number summary and Box plot helped me to understand how to summarize and visualize data distributions effectively. It helps to identify statistics like the median, quartiles and outliers.

~~Repeat~~

Conclusion:

In Conclusion, studying 5 number summary is an exploratory data analysis tool that provides insights into distribution of values for one variable. It is useful in descriptive analysis or during preliminary investigation of large data set. It gives a general sense of whether the distribution is symmetrical or skewed by comparing Q_1 , Q_2 (median), Q_3 .

Assignment 6

Aim: Finding Five number summary for dataset and study of box plot.

1) Using program

Code:

```
import java.io.*;
import java.nio.file.*;
import java.util.*;

public class FiveNumberSummary {

    public static List<String> splitData(String line, char delim) {
        return Arrays.asList(line.split(String.valueOf(delim)));
    }

    public static double findMedian(List<Double> sortedData) {
        int size = sortedData.size();
        if (size % 2 == 0) {
            return (sortedData.get(size / 2 - 1) + sortedData.get(size / 2)) / 2.0;
        } else {
            return sortedData.get(size / 2);
        }
    }

    public static List<Double> calcQuartiles(List<Double> sortedData) {
        List<Double> quarts = new ArrayList<>();
        int size = sortedData.size();

        double med = findMedian(sortedData);

        List<Double> lower = sortedData.subList(0, size / 2);
```



```
double q1 = findMedian(lower);
```

```
List<Double> upper = (size % 2 == 0) ? sortedData.subList(size / 2, size) :
sortedData.subList(size / 2 + 1, size);
```

```
double q3 = findMedian(upper);
```

```
quarts.add(q1);
```

```
quarts.add(med);
```

```
quarts.add(q3);
```

```
return quarts;
```

```
}
```

```
public static void main(String[] args) {
```

```
List<Double> data = new ArrayList<>();
```

```
try (BufferedReader reader =
Files.newBufferedReader(Paths.get("Mine_Dataset_minmax.csv"))) {
```

```
String line = reader.readLine();
```

```
while ((line = reader.readLine()) != null) {
```

```
List<String> fields = splitData(line, ',');
```

```
if (fields.isEmpty()) {
```

```
System.out.println("Empty line encountered, skipping.");
```

```
continue;
```

```
}
```

```
data.add(Double.parseDouble(fields.get(0)));
```

```
}
```

```
Collections.sort(data);
```

```
double minVal = data.get(0);
```

```
double maxVal = data.get(data.size() - 1);
```

```
List<Double> quarts = calcQuartiles(data);
```

```
System.out.println("5-Number Summary:");
```

```

System.out.println("Min: " + minVal);
System.out.println("Q1: " + quarts.get(0));
System.out.println("Median: " + quarts.get(1));
System.out.println("Q3: " + quarts.get(2));
System.out.println("Max: " + maxVal);

} catch (Exception e) {
    System.out.println("Error reading file: " + e.getMessage());
}
}
}

```

```

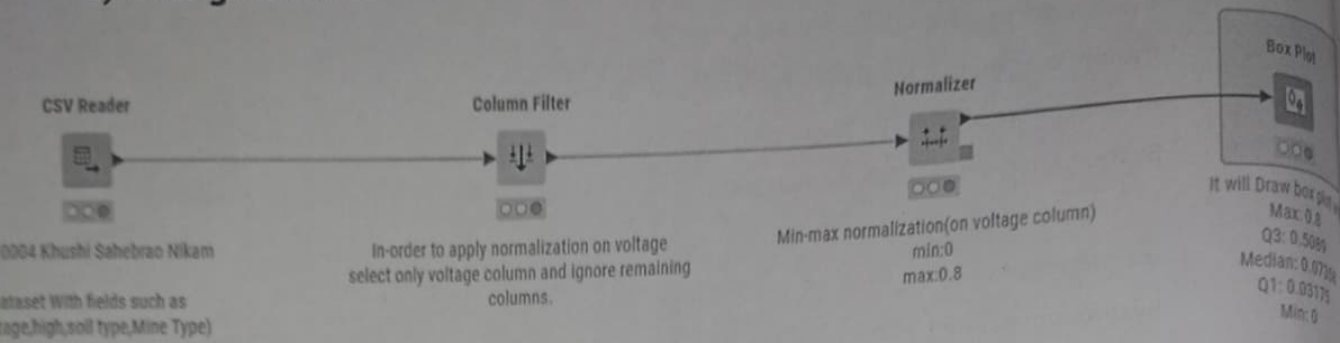
PS C:\Users\khushi\Desktop> javac FiveNumberSummary.java
PS C:\Users\khushi\Desktop> java FiveNumberSummary
5-Number Summary for Voltage:
Min: 0.0
Q1: 0.033282583397331256
Median: 0.07358410705214644
Q3: 0.48285779530399775
Max: 0.8

```

Input (Mine_DataSet.csv):

Voltage	High	Soil Type	Mine Type
0.033283	0.181818	0.6	1
0.033283	0.272727	0.6	1
0.027149	0.454545	0.6	1
0.772119	0.272727	0.6	2
0.58741	0.363636	0.6	2
0	1	1	3
0.068482	0.090909	0.6	1
0.022583	0.363636	0.6	1
0.8	0.181818	0.6	2
0.482858	0.454545	0.6	2
0.263298	0.727273	0.6	2
0.078686	0.545455	0.6	3
0.05719	0.818182	0.6	3
0.102984	0.272727	0.2	3

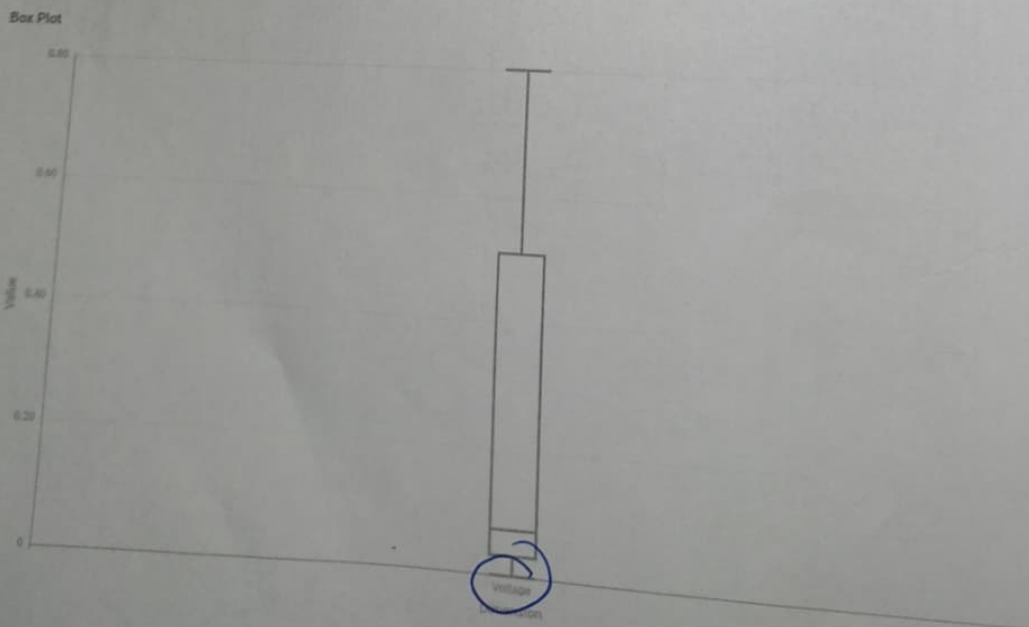
2) Using KNIME:



Input (Mine_DataSet.csv):

Voltage	High	Soil Type	Mine Type
0.033283	0.181818	0.6	1
0.033283	0.272727	0.6	1
0.027149	0.454545	0.6	1
0.772119	0.272727	0.6	2
0.58741	0.363636	0.6	2
0	1	1	3
0.068482	0.090909	0.6	1
0.022583	0.363636	0.6	1
0.8	0.181818	0.6	2
0.482858	0.454545	0.6	2
0.263298	0.727273	0.6	2
0.078686	0.545455	0.6	3
0.05719	0.818182	0.6	3
0.102984	0.272727	0.2	3

Output:





Assignment : 7

Aim: Finding Frequent itemsets on Transaction data.

To find frequent itemsets in a given set of transaction data using association rule mining technique such as the Apriori algorithm or FP-growth algorithm.

Theory :

Frequent itemsets :

The set of items that frequently appear together in a transactional dataset.

Support :

- It is the percentage of transaction in dataset that contain that itemset.
- Used to determine how frequent an itemset within a dataset.

$$\text{support}(x) = \frac{T(x)}{N} \quad \begin{array}{l} \text{— Transaction contains } x \\ \text{— No. of transaction} \end{array}$$

- minimum support threshold : Any itemset that has support greater than or equal to this threshold is considered frequent.

Association rule mining :

- ASR : After identifying frequent itemset, association rules can be generated.
- Common algorithm used to find frequent itemset :



1] Apriori Algorithm :

This uses an iterative process, where larger frequent itemsets are generated from smaller frequent itemsets.

Steps :

1] scan dataset and find frequent item or itemset, where $k+1$ pass uses item or item set found in k pass.

2] The database is scanned and support of candidate is computed.

2] FP-growth algorithm :

Uses tree structure to compress dataset and find frequent itemsets.

Numerical :

TID	Milk	Bread	Butter	Eggs	Apples	Bit Vector
T ₁	Milk	Bread			Apples	1100
T ₂		Bread		Eggs		01010
T ₃		Bread	Butter			01100
T ₄	Milk	Bread		Eggs		11010
T ₅	Milk		Butter			10100
T ₆		Bread	Butter			01100
T ₇	Milk		Butter			10100
T ₈	Milk	Bread	Butter			11101
T ₉	Milk	Bread	Butter		Apples	11100

Minimum Support threshold = 0.3 (30%)



Walchand College of Engineering, Sangli

i) for $k=1$

Itemset	Frequency	Support
{Milk}	6	$(6/9) = 0.67 = 67\%$
{Bread}	7	$(7/9) = 0.78 = 78\%$
{Butter}	6	$(6/9) = 0.67 = 67\%$
{Eggs}	2	$(2/9) = 0.22 = 22\%$
{Apples}	2	$(2/9) = 0.22 = 22\%$

Itemset that meets minimum support threshold
(i.e. ≥ 0.3 (30%)) are
{Milk}, {Bread}, {Butter}

ii) for $k=2$

Itemset	Frequency	Support
{Milk, Bread}	4	$(4/9) = 0.44 = 44\%$
{Milk, Butter}	4	$(4/9) = 0.44 = 44\%$
{Milk, Eggs}	1	$(1/9) = 0.11 = 11\%$
{Milk, Apples}	2	$(2/9) = 0.22 = 22\%$
{Bread, Butter}	4	$(4/9) = 0.44 = 44\%$
{Bread, Eggs}	2	$(2/9) = 0.22 = 22\%$
{Bread, Apples}	2	$(2/9) = 0.22 = 22\%$
{Butter, Eggs}	0	$(0/9) = 0 = 0\%$
{Butter, Apples}	1	$(1/9) = 0.11 = 11\%$
{Eggs, Apples}	0	$(0/9) = 0 = 0\%$

Itemset that meets minimum support threshold
(i.e. ≥ 0.3 (30%)) are
{Milk, Bread}, {Milk, Butter}, {Bread, Butter}



For $k = 3$

Itemset	Frequency	support
{Milk, Bread, Butter}	2	$2/9 = 0.22 = 22\%$
{Milk, Bread, Eggs}	1	$1/9 = 0.11 = 11\%$
{Milk, Bread, Apples}	2	$2/9 = 0.22 = 22\%$
{Milk, Butter, Eggs}	0	$0/9 = 0 = 0\%$
{Milk, Butter, Apples}	1	$1/9 = 0.11 = 11\%$
{Milk, Eggs, Apples}	0	$0/9 = 0 = 0\%$
{Bread, Butter, Eggs}	0	$0/9 = 0 = 0\%$
{Bread, Butter, Apples}	1	$1/9 = 0.11 = 11\%$
{Bread, Eggs, Apples}	0	$0/9 = 0 = 0\%$
{Butter, Eggs, Apples}	0	$0/9 = 0 = 0\%$

Itemsets that meets minimum threshold Values
(i.e ≥ 0.3 (30%))

No itemset found.

$\therefore$ Final Filtered frequent itemsets are

support	Itemsets
0.67 (67%)	{Milk}
0.67 (67%)	{Butter}
0.78 (78%)	{Bread}
0.44 (44%)	{Milk, Bread}
0.44 (44%)	{Milk, Butter}
0.44 (44%)	{Butter, Bread}



Conclusion :

In conclusion, the Apriori algorithm to find frequent item sets has given me understanding of how patterns and relationships in transactional data. This technique is crucial in real-world applications such as market basket analysis, where it helps businesses identify product combinations that are frequently bought together.

Permit

Frequent Itemset mining shows which items appears together in a transaction or relation frequently. It is used to derive relationships such as regularities in customers shopping behavior in physical and online stores to discover association rules out of relationship. It helps to maximize profit and find pattern of customer's area of interest.

Assignment 7

Aim: Finding Frequent Item-set on Transaction Data.

1) Using program

Code:

```
import java.io.*;
import java.util.*;

public class SupportCalculation {

    public static void main(String[] args) {
        String csvFile = "sample.csv";
        String line;
        String csvSplitBy = ",";

        double minSupportThreshold = 0.1;

        List<Set<String>> transactions = new ArrayList<>();

        try (BufferedReader br = new BufferedReader(new FileReader(csvFile))) {
            br.readLine();

            while ((line = br.readLine()) != null) {
                String[] transaction = line.split(csvSplitBy);
                Set<String> items = new HashSet<>();
                for (int i = 1; i < transaction.length; i++) {
                    if (!transaction[i].trim().isEmpty()) {
                        items.add(transaction[i].trim());
                    }
                }
            }
        }
    }
}
```



```

    }
    }
    transactions.add(items);
}

} catch (IOException e) {
    e.printStackTrace();
}

double supportThreshold = minSupportThreshold;
Map<Set<String>, Integer> frequentItemsets = new HashMap<>();
Map<Set<String>, Integer> currentItemsetCount =
getItemsetCounts(transactions, 1);

int k = 1;
while (!currentItemsetCount.isEmpty()) {
    System.out.println("\n" + k + "-itemsets with support >= " +
supportThreshold + ":");
    printFrequentItemsets(currentItemsetCount, transactions.size(),
supportThreshold);
    frequentItemsets.putAll(currentItemsetCount);

    currentItemsetCount = generateNextItemsets(frequentItemsets.keySet(),
transactions, k + 1);
    k++;
}
}

public static Map<Set<String>, Integer> getItemsetCounts(List<Set<String>>
transactions, int setSize) {
    Map<Set<String>, Integer> itemsetCount = new HashMap<>();
    for (Set<String> transaction : transactions) {

```

```

    if (transaction.size() >= setSize) {
        Set<Set<String>> combinations = getCombinations(transaction,
setSize);
        for (Set<String> combination : combinations) {
            itemsetCount.put(combination,
itemsetCount.getDefault(combination, 0) + 1);
        }
    }
}

return itemsetCount;
}

```

```

public static void printFrequentItemsets(Map<Set<String>, Integer>
itemsetCount, int totalTransactions, double minSupportThreshold) {
    for (Map.Entry<Set<String>, Integer> entry : itemsetCount.entrySet()) {
        double support = (double) entry.getValue() / totalTransactions;
        if (support >= minSupportThreshold) {
            System.out.println("Item set: " + entry.getKey() + " | Support: " + support
+ " | Count: " + entry.getValue());
        }
    }
}

```

```

public static Set<Set<String>> getCombinations(Set<String> transaction, int
setSize) {
    Set<Set<String>> combinations = new HashSet<>();
    List<String> transactionList = new ArrayList<>(transaction);
    generateCombinations(combinations, new HashSet<>(), transactionList, 0,
setSize);
    return combinations;
}

```

```

public static void generateCombinations(Set<Set<String>> combinations,
Set<String> currentSet, List<String> items, int index, int setSize) {
    if (currentSet.size() == setSize) {
        combinations.add(new HashSet<>(currentSet));
        return;
    }
    for (int i = index; i < items.size(); i++) {
        currentSet.add(items.get(i));
        generateCombinations(combinations, currentSet, items, i + 1, setSize);
        currentSet.remove(items.get(i));
    }
}

```

```

public static Map<Set<String>, Integer>
generateNextItemsets(Set<Set<String>> previousItemsets, List<Set<String>>
transactions, int setSize) {
    Map<Set<String>, Integer> nextItemsetCount = new HashMap<>();
    for (Set<String> transaction : transactions) {
        if (transaction.size() >= setSize) {
            Set<Set<String>> combinations = getCombinations(transaction,
setSize);
            for (Set<String> combination : combinations) {
                nextItemsetCount.put(combination,
nextItemsetCount.getOrDefault(combination, 0) + 1);
            }
        }
    }
    return nextItemsetCount;
}
}

```



```
PS C:\Users\khushi\Desktop> javac SupportCalculation.java
PS C:\Users\khushi\Desktop> java SupportCalculation
```

1-itemsets with support ≥ 0.3 :

Item set: [Butter] | Support: 0.6666666666666666 | Count: 6

Item set: [Milk] | Support: 0.6666666666666666 | Count: 6

Item set: [Bread] | Support: 0.7777777777777778 | Count: 7

2-itemsets with support ≥ 0.3 :

Item set: [Butter, Milk] | Support: 0.4444444444444444 | Count: 4

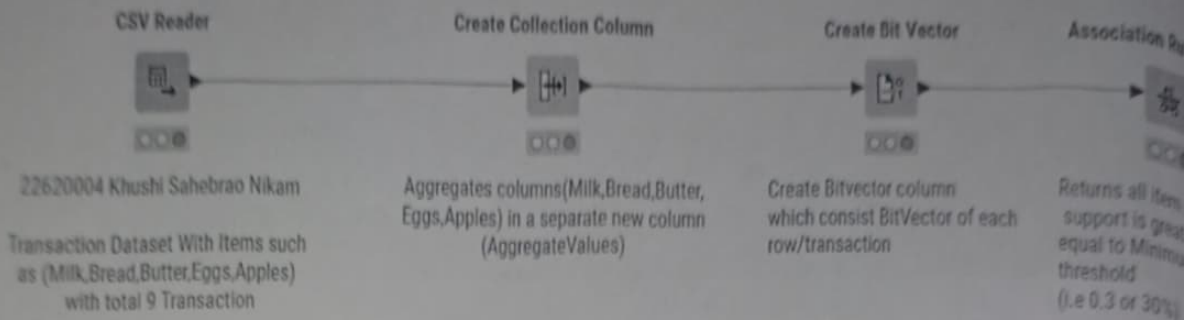
Item set: [Butter, Bread] | Support: 0.4444444444444444 | Count: 4

Item set: [Milk, Bread] | Support: 0.4444444444444444 | Count: 4

Input (Mine_DataSet.csv):

TID	Milk	Bread	Butter	Eggs	Apples
T1	Milk	Bread			Apples
T2		Bread		Eggs	
T3		Bread	Butter		
T4	Milk	Bread		Eggs	
T5	Milk		Butter		
T6		Bread	Butter		
T7	Milk		Butter		
T8	Milk	Bread	Butter		Apples
T9	Milk	Bread	Butter		

2) Using KNIME:



TID	Milk	Bread	Butter	Eggs	Apples
T1	Milk	Bread			Apples
T2		Bread		Eggs	
T3		Bread	Butter		
T4	Milk	Bread		Eggs	
T5	Milk		Butter		
T6		Bread	Butter		
T7	Milk		Butter		
T8	Milk	Bread	Butter		Apples
T9	Milk	Bread	Butter		

Input (Sample.csv):

Output:

► 1: Frequent Itemsets/Association rules ☐ Flow Variables

Rows: 6 | Columns: 2

Table ☒ Statistics ☐

☐	#	RowID	Support(0-1): Number (Double)	Items Set
☐	1	Item ...	0.444	[Milk,Bread]
☐	2	Item ...	0.444	[Butter,Milk]
☐	3	Item ...	0.444	[Butter,Bread]
☐	4	Item ...	0.667	[Milk]
☐	5	Item ...	0.667	[Butter]
☐	6	Item ...	0.778	[Bread]

3) Weka Output:

```
*** Run information ***
```

```
Scheme:      weka.associations.Apriori -N 10 -T 0 -C 0.9 -O 0.05 -U 1.0 -M 0.1 -S -1.0 -c -1
Relation:     sample
Instances:    9
Attributes:   6
              TID
              Milk
              Bread
              Butter
              Eggs
              Apples
```

```
=== Associator model (full training set) ===
```

```
Apriori
```

```
Minimum support: 0.16 (1 instances)
Minimum metric <confidence>: 0.9
Number of cycles performed: 17
```

```
Generated sets of large itemsets:
```

```
Size of set of large itemsets L(1): 14
```

```
Size of set of large itemsets L(2): 31
```

```
Size of set of large itemsets L(3): 25
```

```
Size of set of large itemsets L(4): 8
```

```
Size of set of large itemsets L(5): 1
```

```
Best rules found:
```

1. Apples=Apples 2 ==> Milk=Milk 2 <conf:(1)> lift:(1.5) lev:(0.07) [0] conv:(0.67)
2. Eggs=Eggs 2 ==> Bread=Bread 2 <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
3. Apples=Apples 2 ==> Bread=Bread 2 <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
4. Bread=Bread Apples=Apples 2 ==> Milk=Milk 2 <conf:(1)> lift:(1.5) lev:(0.07) [0] conv:(0.67)
5. Milk=Milk Apples=Apples 2 ==> Bread=Bread 2 <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
6. Apples=Apples 2 ==> Milk=Milk Bread=Bread 2 <conf:(1)> lift:(2.25) lev:(0.12) [1] conv:(1.11)
7. TID=T1 1 ==> Milk=Milk 1 <conf:(1)> lift:(1.5) lev:(0.04) [0] conv:(0.33)
8. TID=T1 1 ==> Bread=Bread 1 <conf:(1)> lift:(1.29) lev:(0.02) [0] conv:(0.22)
9. TID=T1 1 ==> Apples=Apples 1 <conf:(1)> lift:(4.5) lev:(0.09) [0] conv:(0.78)
10. TID=T2 1 ==> Bread=Bread 1 <conf:(1)> lift:(1.29) lev:(0.02) [0] conv:(0.22)



Assignment 8

Aim: Extend program 7, to find association rule.

Theory:

Association rule mining is a fundamental technique in data mining that uncovers interesting relationships between variables in large datasets. The primary goals of this technique are to identify frequent itemsets and derive association rule from them.

* Support:

Proportion of transactions in the dataset that contain the itemset.

$$\text{Support}(A) = \frac{\text{count}(A)}{\text{Total transaction}}$$

* Confidence:

It is measure of the likelihood that an item Y is also purchased when item X is purchased.

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \cup Y)}{\text{Support}(X)}$$

Methodology:

- 1) Take user i/p for minimum support and minimum confidence
- 2) Generate frequent itemset using Apriori algorithm
- 3) Generate association rules for all generated frequent itemsets that meet the minimum confidence threshold.



Walchand College Of Engineering, Sangli.

Numerical :

minimum support (0.3 / 30%)

thus frequent itemset got from program?

- { Butter } $\rightarrow$ support (count) = 6
- { Milk } $\rightarrow$ support (count) = 6
- { Bread } $\rightarrow$ support (count) = 6
- { Butter, Milk } $\rightarrow$ support (count) = 4
- { Butter, Bread } $\rightarrow$ support (count) = 4
- { Milk, Bread } $\rightarrow$ support (count) = 4

Rule	Support (count)	Confidence	Confidence
Milk $\rightarrow$ Bread	4	4/6	66.67%
Bread $\rightarrow$ Milk	4	4/6	66.67%
Milk $\rightarrow$ Butter	4	4/6	66.67%
Butter $\rightarrow$ Milk	4	4/6	66.67%
Bread $\rightarrow$ Butter	4	4/6	66.67%
Butter $\rightarrow$ Bread	4	4/6	66.67%

Generated Association Rules with
minimum Confidence of 50% $\Rightarrow$

- ① Milk $\rightarrow$ Bread
- ② Bread $\rightarrow$ Milk
- ③ Butter $\rightarrow$ Milk
- ④ Bread $\rightarrow$ Butter
- ⑤ Butter $\rightarrow$ Bread
- ⑥ Milk $\rightarrow$ Butter



Conclusion :

The process of finding association rule through the Apriori enables us to find significant relationships between items in datasets. By calculating support & confidence we identified multiple strong association rules that meets the defined thresholds. This analysis is particularly beneficial in market basket analysis, for understanding customer purchasing behavior can guide inventory management.

Assignment 8

Aim: Extend program 7, to find association rule.

1) Using program

Code:

```
import java.io.*;
import java.util.*;

public class AssociationRuleMining {

    public static void main(String[] args) {
        String csvFile = "sample.csv";
        List<Set<String>> transactions = new ArrayList<>();

        try (BufferedReader br = new BufferedReader(new FileReader(csvFile))) {
            br.readLine(); // Skip header
            String line;
            while ((line = br.readLine()) != null) {
                String[] transaction = line.split(",");
                Set<String> items = new HashSet<>();
                for (int i = 1; i < transaction.length; i++) {
                    if (!transaction[i].trim().isEmpty()) {
                        items.add(transaction[i].trim());
                    }
                }
                transactions.add(items);
            }
        } catch (IOException e) {
```

```

        e.printStackTrace();
    }

    Scanner scanner = new Scanner(System.in);
    System.out.print("Enter Minimum Support Threshold (as fraction): ");
    double minSupportThreshold = scanner.nextDouble();

    System.out.print("Enter Minimum Confidence Threshold (as fraction): ");
    double minConfidenceThreshold = scanner.nextDouble();
    scanner.close();

    int totalTransactions = transactions.size();
    Map<Set<String>, Integer> frequentItemsets = new HashMap<>();
    Map<Set<String>, Integer> currentItemsetCount =
        getItemsetCounts(transactions, 1);

    int k = 1;
    while (!currentItemsetCount.isEmpty()) {
        System.out.printf("\n%d-itemsets with support >= %.2f:\n", k,
            minSupportThreshold);

        System.out.printf("| %-20s | %-10s | %-10s |\n", "Itemset", "Count",
            "Support (%)");

        System.out.println("-----");

        filterAndPrintFrequentItemsets(currentItemsetCount, totalTransactions,
            minSupportThreshold, frequentItemsets);

        currentItemsetCount = generateNextItemsets(frequentItemsets.keySet(),
            transactions, k + 1);

        k++;
    }

    System.out.printf("\nAssociation Rules with Confidence >= %.2f:\n",
        minConfidenceThreshold);

```

```
System.out.printf("| %-20s | %-20s | %-10s | \n", "Rule", "Confidence",
"Confidence (%)");
```

```
System.out.println("-----");
```

```
findAssociationRules(frequentItemsets, totalTransactions,
minConfidenceThreshold);
```

```
}
```

```
public static Map<Set<String>, Integer> getItemsetCounts(List<Set<String>>
transactions, int setSize) {
```

```
Map<Set<String>, Integer> itemsetCount = new HashMap<>();
```

```
for (Set<String> transaction : transactions) {
```

```
if (transaction.size() >= setSize) {
```

```
Set<Set<String>> combinations = getCombinations(transaction,
setSize);
```

```
for (Set<String> combination : combinations) {
```

```
itemsetCount.put(combination,
itemsetCount.getDefault(combination, 0) + 1);
```

```
}
```

```
}
```

```
}
```

```
return itemsetCount;
```

```
}
```

```
public static void filterAndPrintFrequentItemsets(Map<Set<String>, Integer>
itemsetCount, int totalTransactions, double minSupportThreshold,
Map<Set<String>, Integer> frequentItemsets) {
```

```
Iterator<Map.Entry<Set<String>, Integer>> iterator =
itemsetCount.entrySet().iterator();
```

```
while (iterator.hasNext()) {
```

```
Map.Entry<Set<String>, Integer> entry = iterator.next();
```

```
double support = (double) entry.getValue() / totalTransactions;
```

```
if (support >= minSupportThreshold) {
```

```
frequentItemsets.put(entry.getKey(), entry.getValue());
```



```

        System.out.printf("| %-20s | %-10d | %-10.2f | \n", entry.getKey(),
entry.getValue(), support * 100);
    } else {
        iterator.remove();
    }
}
}
}

```

```

public static Set<Set<String>> getCombinations(Set<String> transaction, int
setSize) {
    Set<Set<String>> combinations = new HashSet<>();
    List<String> transactionList = new ArrayList<>(transaction);
    generateCombinations(combinations, new HashSet<>(), transactionList, 0,
setSize);
    return combinations;
}

```

```

public static void generateCombinations(Set<Set<String>> combinations,
Set<String> currentSet, List<String> items, int index, int setSize) {
    if (currentSet.size() == setSize) {
        combinations.add(new HashSet<>(currentSet));
        return;
    }
    for (int i = index; i < items.size(); i++) {
        currentSet.add(items.get(i));
        generateCombinations(combinations, currentSet, items, i + 1, setSize);
        currentSet.remove(items.get(i));
    }
}
}

```

```

public static Map<Set<String>, Integer>
generateNextItemsets(Set<Set<String>> previousItemsets, List<Set<String>>
transactions, int setSize) {
    Map<Set<String>, Integer> nextItemsetCount = new HashMap<>();
    for (Set<String> transaction : transactions) {
        if (transaction.size() >= setSize) {
            Set<Set<String>> combinations = getCombinations(transaction,
setSize);
            for (Set<String> combination : combinations) {
                if (allSubsetsAreFrequent(combination, previousItemsets)) {
                    nextItemsetCount.put(combination,
nextItemsetCount.getDefault(combination, 0) + 1);
                }
            }
        }
    }
    return nextItemsetCount;
}

public static boolean allSubsetsAreFrequent(Set<String> combination,
Set<Set<String>> previousItemsets) {
    for (String item : combination) {
        Set<String> subset = new HashSet<>(combination);
        subset.remove(item);
        if (!previousItemsets.contains(subset)) {
            return false;
        }
    }
    return true;
}

public static void findAssociationRules(Map<Set<String>, Integer>
frequentItemsets, int totalTransactions, double minConfidenceThreshold) {
    for (Map.Entry<Set<String>, Integer> entry : frequentItemsets.entrySet()) {
        Set<String> itemset = entry.getKey();
        int itemsetCount = entry.getValue();
        if (itemset.size() > 1) {
            Set<Set<String>> subsets = getCombinations(itemset, itemset.size() - 1);

```



```
for (Set<String> subset : subsets) {
    Set<String> remaining = new HashSet<>(itemset);
    remaining.removeAll(subset);
    int subsetCount = frequentItemsets.getDefault(subset, 0);
    if (subsetCount > 0) {
        double confidence = (double) itemsetCount / subsetCount;
        if (confidence >= minConfidenceThreshold) {
            System.out.printf("| %-20s | %-20.3f | %-10.2f | \n", subset + " =>
" + remaining, confidence, confidence * 100);
        }
    }
}
```

```
PS C:\Users\khushi\Desktop\DM\Assignment8> java AssociationRuleMining
Enter Minimum Support Threshold (as fraction): 0.3
Enter Minimum Confidence Threshold (as fraction): 0.5
```

```
1-itemsets with support >= 0.30:
```

Itemset	Count	Support (%)
[Butter]	6	66.67
[Milk]	6	66.67
[Bread]	7	77.78

2-itemsets with support ≥ 0.30 :

Itemset	Count	Support (%)
[Butter, Milk]	4	44.44
[Butter, Bread]	4	44.44
[Milk, Bread]	4	44.44

3-itemsets with support ≥ 0.30 :

Itemset	Count	Support (%)
---------	-------	-------------

Association Rules with Confidence ≥ 0.50 :

Rule	Confidence	Confidence (%)
[Butter] => [Milk]	0.667	66.67
[Milk] => [Butter]	0.667	66.67
[Butter] => [Bread]	0.667	66.67
[Bread] => [Butter]	0.571	57.14
[Milk] => [Bread]	0.667	66.67
[Bread] => [Milk]	0.571	57.14

Name: Nikam Khushi Sahebrao Nikam[22620004]

TID	Milk	Bread	Butter	Eggs	Apples
T1	Milk	Bread			Apples
T2		Bread		Eggs	
T3		Bread	Butter		
T4	Milk	Bread		Eggs	
T5	Milk		Butter		
T6		Bread	Butter		
T7	Milk		Butter		
T8	Milk	Bread	Butter		Apples
T9	Milk	Bread	Butter		

Input (region_product_data.csv):

Weka:

Weka Explorer

Preprocess Classify Cluster Associate Select attributes Visualize

Associate

Choose Apriori N 10 T 0 C 0.9 D 0.05 U 1.0 M 0.1 S 1.0 c 1

Start

Associate output

Result list (right-click...)

11:38:53 - Apriori

eggs
Apples
--- Associate model (full training set) ---

Apriori

Minimum support: 0.16 (1 instances)

Minimum metric <confidence>: 0.9

Number of cycles performed: 17

Generated sets of large itemsets:

Size of set of large itemsets L(1): 14

Size of set of large itemsets L(2): 31

Size of set of large itemsets L(3): 25

Size of set of large itemsets L(4): 8

Size of set of large itemsets L(5): 1

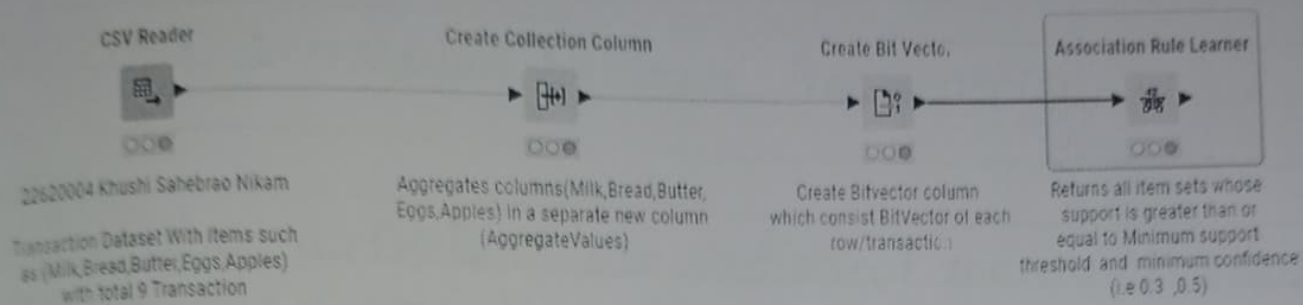
Best rules found:

1. Apples=Apples 2 ==> Milk=Milk 2 <conf:(1)> lift:(1.5) lev:(0.07) [0] conv:(0.67)
2. Eggs=Eggs 2 ==> Bread=Bread 2 <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
3. Apples=Apples 2 ==> Bread=Bread 2 <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
4. Bread=Bread Apples=Apples 2 ==> Milk=Milk 2 <conf:(1)> lift:(1.5) lev:(0.07) [0] conv:(0.67)
5. Milk=Milk Apples=Apples 2 ==> Bread=Bread 2 <conf:(1)> lift:(1.29) lev:(0.05) [0] conv:(0.44)
6. Apples=Apples 2 ==> Milk=Milk Bread=Bread 2 <conf:(1)> lift:(2.25) lev:(0.12) [1] conv:(1.11)
7. TID=T1 1 ==> Milk=Milk 1 <conf:(1)> lift:(1.5) lev:(0.04) [0] conv:(0.33)
8. TID=T1 1 ==> Bread=Bread 1 <conf:(1)> lift:(1.29) lev:(0.02) [0] conv:(0.22)
9. TID=T1 1 ==> Apples=Apples 1 <conf:(1)> lift:(4.5) lev:(0.09) [0] conv:(0.78)
10. TID=T2 1 ==> Bread=Bread 1 <conf:(1)> lift:(1.29) lev:(0.02) [0] conv:(0.22)

Status

OK

Log

KNIME:**Output:**

Rows: 6 | Columns: 6

Table Statistics

#	RowID	Support Number (double)	Confidence Number (double)	Lift Number (double)	Consequent String	implies String	Items Set
1	rule0	0.444	0.571	0.857	Milk	<---	[Bread]
2	rule1	0.444	0.667	0.857	Bread	<---	[Milk]
3	rule2	0.444	0.667	1	Milk	<---	[Butter]
4	rule3	0.444	0.667	1	Butter	<---	[Milk]
5	rule4	0.444	0.667	0.857	Bread	<---	[Butter]
6	rule5	0.444	0.571	0.857	Butter	<---	[Bread]



Assignment : 9

Aim: Find Correlation between items/entities.

Theory :

Correlation coefficient are statistical measures that express the degree to which two variables are related. They range from -1 to 1, indicating the strength and direction of relationship between the variables.

- * Positive Correlation: A value close to 1 indicates a strong positive relationships, meaning that as one variable increase, the other also tends to increase.
- * Negative Correlation: A value close to -1 indicates a strong Negative relationships, meaning that variable tends to decrease as the other increases.
- * No correlation: A value around 0 indicates no relationship between the variables.

$$C(A,B) = \frac{P(A \cap B)}{P(A) \cdot P(B)}$$

where, $C(A,B) \rightarrow$ correlation coefficient betⁿ A & B.
 $P(A \cap B) \rightarrow$ Probability of both item occurring together.
 $P(A) \rightarrow$ probability of item A occurring
 $P(B) \rightarrow$ Probability of item B occurring.



Numerical :

correlation calculation :

1) Sports-Music =

$$C(S, M) = \frac{2}{3 \times 3} = 0.22 \quad (+ve)$$

2) Sports-Dance =

$$C(S, D) = \frac{1}{3 \times 3} = 0.11 \quad (+ve)$$

3) Sports-Art =

$$C(S, A) = \frac{1}{3 \times 2} = 0.17 \quad (+ve)$$

4) Music-Dance =

$$C(M, D) = \frac{1}{3 \times 3} = 0.11 \quad (+ve)$$

5) Music-Art

$$C(M, A) = \frac{2}{3 \times 2} = 0.33 \quad (+ve)$$

6) Dance-Art

$$C(D, A) = \frac{1}{3 \times 2} = 0.17 \quad (+ve)$$

Conclusion :

Calculating correlation ratios provides valuable insights into the relationships between different items or entities. By applying the correlation coefficient formula, we were able to assess the strength and direction of associations, highlighting areas of interest in student activities. It helps to make data-driven decisions and improve overall effectiveness.

Assignment 9

Aim: Find correlation between items/entities.

1) Using program

Code:

```
import java.io.*;
import java.util.*;
import java.util.stream.*;

public class Correlation {

    public static void main(String[] args) throws Exception {

        String inputFile = "student_data.csv";
        List<String[]> data = readCSV(inputFile);

        int totalRows = data.size() - 1;
        List<String[]> results = new ArrayList<>();

        results.add(new String[]{"Item 1 (Activity)", "Item 2 (Activity)", "Pearson  
Correlation Coefficient", "Type of Correlation"});

        for (int i = 1; i <= totalRows; i++) {
            for (int j = i + 1; j <= totalRows; j++) {
                double correlation = findPearsonCorrelation(data, i, j);
                String verdict = getVerdict(correlation);
                String item1 = data.get(i)[0];
                String item2 = data.get(j)[0];
                results.add(new String[]{item1, item2, String.format("%.3f", correlation),  
verdict});
            }
        }
    }
}
```



```
writeCSV("PearsonCorrelation_output.csv", results);  
}
```

```
public static List<String[]> readCSV(String fileName) throws IOException {  
    List<String[]> data = new ArrayList<>();  
    try (BufferedReader br = new BufferedReader(new FileReader(fileName))) {  
        String line;  
        while ((line = br.readLine()) != null) {  
            String[] values = line.split(",");  
            data.add(values);  
        }  
    }  
    return data;  
}
```

```
public static void writeCSV(String fileName, List<String[]> data) throws  
IOException {  
    try (BufferedWriter bw = new BufferedWriter(new FileWriter(fileName))) {  
        for (String[] row : data) {  
            bw.write(String.join(",", row));  
            bw.newLine();  
        }  
    }  
}
```

```
public static double findPearsonCorrelation(List<String[]> data, int tid1, int  
tid2) {  
    List<Integer> x = new ArrayList<>();  
    List<Integer> y = new ArrayList<>();
```

```

for (int j = 1; j < data.get(0).length; j++) {
    x.add(Integer.parseInt(data.get(tid1)[j]));
    y.add(Integer.parseInt(data.get(tid2)[j]));
}

```

```

int n = x.size();

```

```

int sumX = x.stream().mapToInt(Integer::intValue).sum();

```

```

int sumY = y.stream().mapToInt(Integer::intValue).sum();

```

```

int sumX2 = x.stream().mapToInt(val -> val * val).sum();

```

```

int sumY2 = y.stream().mapToInt(val -> val * val).sum();

```

```

int sumXY = IntStream.range(0, n).map(i -> x.get(i) * y.get(i)).sum();

```

```

double numerator = n * sumXY - sumX * sumY;

```

```

double denominator = Math.sqrt((n * sumX2 - sumX * sumX) * (n * sumY2 -
sumY * sumY));

```

```

return (denominator == 0) ? 0 : numerator / denominator;

```

```

}

```

```

public static String getVerdict(double correlation) {

```

```

    if (correlation == 0) return "No relationship";

```

```

    if (correlation > 0) return "Positive correlation";

```

```

    return "Negative correlation";

```

```

}

```

```

}

```

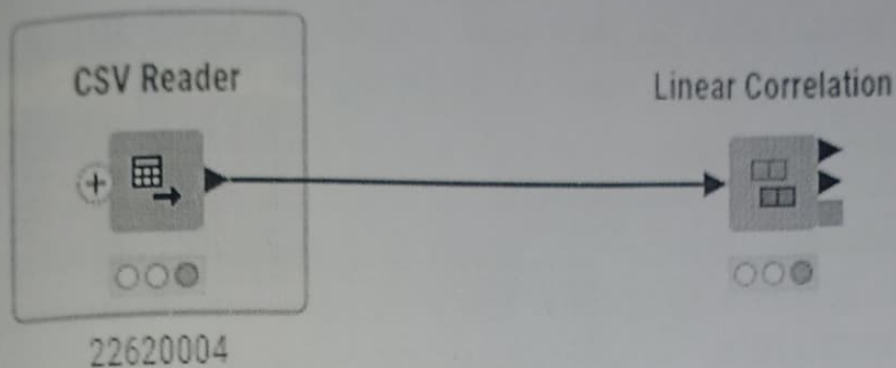
Input (student_data.csv):

Activity	Student1	Student2	Student3	Student4	Student5
Running	1	0	1	1	0
Swimming	0	1	1	0	1
Cycling	1	0	0	1	1
Yoga	0	1	0	1	0

Output(correlation_output.csv):

Item 1 (Ac	Item 2 (Ac	Pearson C	Type of Correlation
Running	Swimming	-0.667	Negative correlation
Running	Cycling	0.167	Positive correlation
Running	Yoga	-0.167	Negative correlation
Swimming	Cycling	-0.667	Negative correlation
Swimming	Yoga	-0.167	Negative correlation
Cycling	Yoga	-0.167	Negative correlation

KNIME:



Output:

#	RowID	First column na... String	Second column ... String	Correlation ... Number (double)	p value Number (double)	Degrees of free... Number (integer)	
☐	5	Row4	Music (M)	Art (A)	0.667	0.219	3
☐	1	Row0	Sports (S)	Music (M)	0.167	0.789	3
☐	6	Row5	Dance (D)	Art (A)	-0.167	0.789	3
☐	3	Row2	Sports (S)	Art (A)	-0.167	0.789	3
☐	2	Row1	Sports (S)	Dance (D)	-0.667	0.219	3
☐	4	Row3	Music (M)	Dance (D)	-0.667	0.219	3

Weka:

Weka Explorer

Preprocess Classify Cluster Associate Select attributes Visualize

Attribute Evaluator
Choose **CorrelationAttributeEval**

Search Method
Choose **Ranker -T -1.7976931348623157E308 -N -1**

Attribute Selection Mode
☒ Use full training set
☐ Cross-validation Folds 10 Seed 1

No class

Start Stop

Result list (right-click for options)
11:44:56 - Ranker + CorrelationAttributeEval

Attribute selection output

```

=== Run information ===
Evaluator:   weka.attributeSelection.CorrelationAttributeEval
Search:     weka.attributeSelection.Ranker -T -1.7976931348623157E308 -N -1
Relation:   student_activities
Instances:  5
Attributes: 5
            Student
            Sports (S)
            Music (M)
            Dance (D)
            Art (A)
Evaluation mode: evaluate on all training data

=== Attribute Selection on all input data ===
Search Method:
  Attribute ranking.

Attribute Evaluator (supervised, Class (numeric): 5 Art (A)):
  Correlation Ranking Filter

Ranked attributes:
  0.667 3 Music (M)
  0.49  1 Student
  -0.167 4 Dance (D)
  -0.167 2 Sports (S)

Selected attributes: 3,1,4,2 : 4
  
```

Status
OK

Log x0



Walchand College Of Engineering, Sangli.

Assignment : 10

Aim : unsupervised learning methods - cluster analysis - partition-based clustering.

Theory :

Partition-based clustering divides a dataset into distinct clusters based on pre-defined criteria, usually either K-means or K-Medoid methods. These algorithm partition the data points into 'k' clusters by minimizing the distance between the data points and the cluster center (for K-means) or a representative data point called a medoid (for K-medoid).

• K-Means Clustering :

It is an iterative algorithm that aims to partition 'n' data points into 'k' clusters where each data point belongs to the cluster with the nearest mean (centroid).

Steps :

1. Select value of k.
2. Choose k initial points from the data as initial centroid.
3. Compute the distance between all data points and the centroid.
4. Assign each data point to the cluster whose centroid is closest.
5. Recalculate the centroids based on the current assignment of points.
6. Repeat steps 3-5 until convergence.

• K-Medoid clustering :

Partition based algorithm based on clustering technique similar to K-means. However, instead of using centroids, it selects actual data points as the cluster center.



Numerical :

a - 0.8853

b - 0.3363

c - 0.3065

d - 0.9999

e - 0.5347

⇒ No. of clusters = 2 (i.e. $k=2$)

⇒ Initialize centroids (Random selection)

$C_1 = 0.3353$

$C_2 = 0.9999$

⇒

	C_1	C_2	min distance	Close to
(0.3353) a	0	0.6646	0	C_1
(0.3363) b	0.001	0.6636	0.001	C_1
(0.3065) c	0.0288	0.6934	0.0288	C_1
(0.9999) d	0.6646	0	0	C_2
(0.5347) e	0.1994	0.4652	0.1994	C_1

∴ cluster 1 = { 0.3353, 0.3363, 0.3065, 0.5347 }
 cluster 2 = { 0.9999 } ,

⇒ Compute New centroids :

$$C_1 \text{ (New centroid)} = \frac{0.3353 + 0.3363 + 0.3065 + 0.5347}{4}$$

$$= \frac{1.5128}{4}$$

$C_1 = 0.3782$

$C_2 \text{ (New } C_2) = 0.9999$

$C_2 = 0.9999$



Walchand College Of Engineering, Sangli.

	0.3782	0.9999		
	c_1	c_2	min distance	close to
(0.3353) a	0.0429	0.6646	0.0429	c_1
(0.3363) b	0.0419	0.6636	0.0419	c_1
(0.3065) c	0.0717	0.6934	0.0717	c_1
(0.9999) d	0.6217	0	0	c_2
(0.5347) e	0.1565	0.4652	0.1565	c_1

$\therefore$ cluster 1 = {0.3353, 0.3363, 0.3065, 0.5347}
cluster 2 = {0.9999}

→ Cluster assignments have not changed after last iteration, so algorithm has converged.

→ Final Clusters:

cluster 1 = {0.3353, 0.3363, 0.3065, 0.5347}
cluster 2 = {0.9999}

Conclusion

In conclusion, k-means method for partitioned-based clustering has provided me knowledge of how to group data into distinct clusters. This method is particularly useful for large datasets, as it helps to identify patterns and similarities among data points.

Reed



Conclusion :

k-means clustering partitions dataset into k-predefined distinct non-overlapping subgroups (clusters) where each point belongs to any one group. Clustering by k-means helps to identify patterns and similarities among large datasets.

Assignment 10

Aim: Unsupervised Learning Methods: Cluster Analysis - partitioning based.

1) Using program

Code:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class KMeansClustering {
    public static void main(String[] args) {
        String csvFile = "Mine_Data.csv";
        List<Double> voltageValues = new ArrayList<>();
        Scanner scanner = new Scanner(System.in);

        try (BufferedReader br = new BufferedReader(new FileReader(csvFile))) {
            String line;
            br.readLine();
            while ((line = br.readLine()) != null) {
                String[] fields = line.split(",");
                double voltage = Double.parseDouble(fields[0]);
                voltageValues.add(voltage);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

}

```
System.out.print("Enter the number of clusters (K): ");
int k = scanner.nextInt();
```

```
List<Double> centroids = new ArrayList<>();
```

```
for (int i = 0; i < k; i++) {
```

```
    System.out.print("Enter initial centroid " + (i + 1) + ": ");
```

```
    centroids.add(scanner.nextDouble());
```

```
}
```

```
List<Integer> clusterAssignments = new ArrayList<>();
```

```
boolean centroidsChanged = true;
```

```
int iterations = 0;
```

```
while (centroidsChanged && iterations < 100) {
```

```
    iterations++;
```

```
    centroidsChanged = false;
```

```
    clusterAssignments.clear();
```

```
    for (double value : voltageValues) {
```

```
        int nearestCentroidIndex = findNearestCentroid(value, centroids);
```

```
        clusterAssignments.add(nearestCentroidIndex);
```

```
    }
```

```
    for (int i = 0; i < k; i++) {
```

```
        List<Double> clusterPoints = getClusterPoints(voltageValues,
clusterAssignments, i);
```

```
        if (!clusterPoints.isEmpty()) {
```

```
            double newCentroid = calculateMean(clusterPoints);
```

```
            if (Math.abs(centroids.get(i) - newCentroid) > 1e-4) {
```

```
                centroids.set(i, newCentroid);
```

```
centroidsChanged = true;
```

```
}
```

```
}
```

```
}
```

```
System.out.println("Iteration " + iterations + ": Centroids = " + centroids);
```

```
}
```

```
System.out.println("Final Cluster Assignments:");
```

```
for (int i = 0; i < voltageValues.size(); i++) {
```

```
    System.out.println("Value: " + voltageValues.get(i) + " -> Cluster: " +  
clusterAssignments.get(i));
```

```
}
```

```
}
```

```
private static int findNearestCentroid(double value, List<Double> centroids) {
```

```
    int nearestCentroidIndex = 0;
```

```
    double minDistance = Math.abs(value - centroids.get(0));
```

```
    for (int i = 1; i < centroids.size(); i++) {
```

```
        double distance = Math.abs(value - centroids.get(i));
```

```
        if (distance < minDistance) {
```

```
            minDistance = distance;
```

```
            nearestCentroidIndex = i;
```

```
        }
```

```
    }
```

```
    return nearestCentroidIndex;
```

```
}
```

```
private static List<Double> getClusterPoints(List<Double> data, List<Integer>  
assignments, int clusterIndex) {
```

```
    List<Double> clusterPoints = new ArrayList<>();
```



```

for (int i = 0; i < data.size(); i++) {
    if (assignments.get(i) == clusterIndex) {
        clusterPoints.add(data.get(i));
    }
}
return clusterPoints;
}
private static double calculateMean(List<Double> points) {
    double sum = 0;
    for (double point : points) {
        sum += point;
    }
    return sum / points.size();
}
}

```

Input (Mine_Data.csv):

Voltage	High	Soil Type	Mine Type
0.335347	0.181818	0.6	1
0.336347	0.272727	0.6	1
0.306495	1	1	3
0.999999	0.181818	0.6	2
0.534743	0.727273	0.6	2

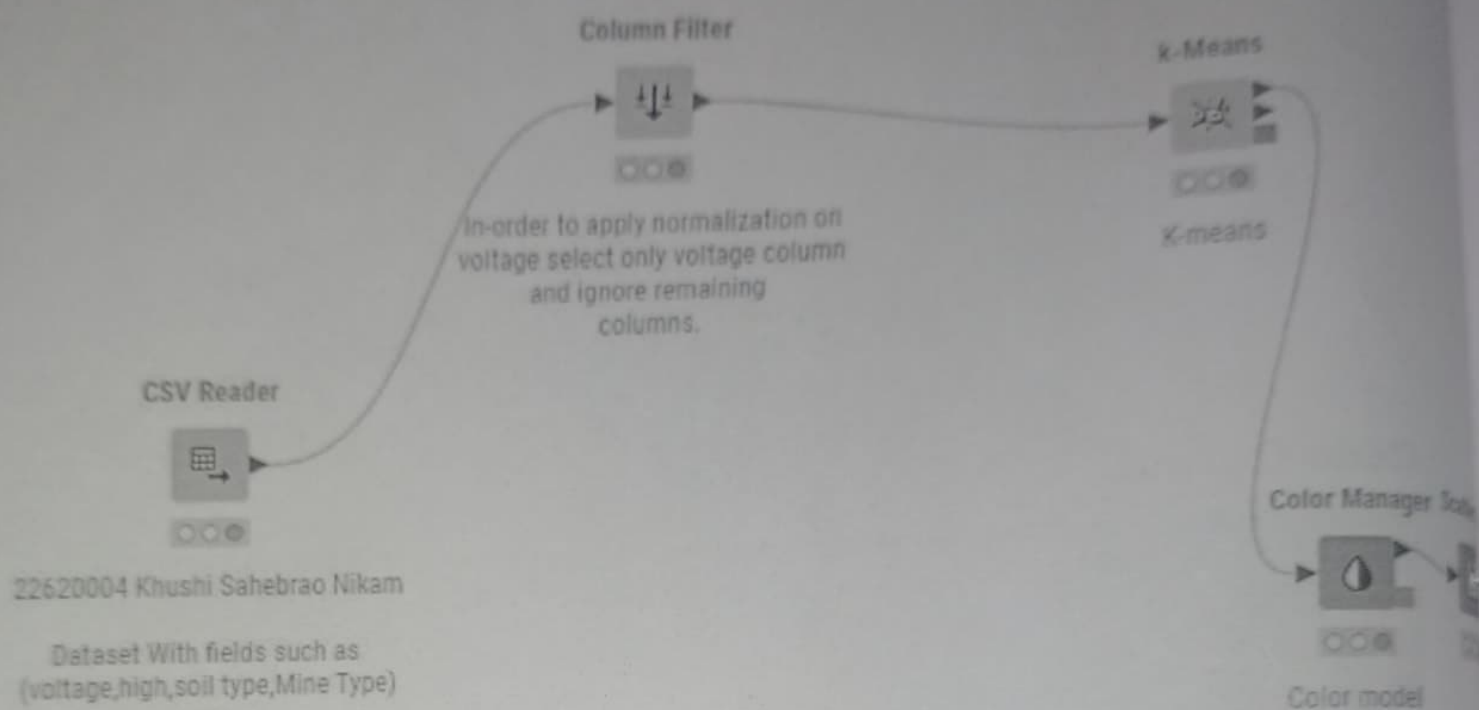
Code Output:

```

PS C:\Users\khushi\Desktop\DM> javac .\KMeansClustering.java
PS C:\Users\khushi\Desktop\DM> java KMeansClustering
Enter the number of clusters (K): 2
Enter initial centroid 1: 0.3353
Enter initial centroid 2: 0.9999
Iteration 1: Centroids = [0.3782329315, 0.9999]
Iteration 2: Centroids = [0.3782329315, 0.9999]
Final Cluster Assignments:
Value: 0.335347054 -> Cluster: 0
Value: 0.336347054 -> Cluster: 0
Value: 0.306495062 -> Cluster: 0
Value: 0.999998728 -> Cluster: 1
Value: 0.534742556 -> Cluster: 0
PS C:\Users\khushi\Desktop\DM>

```

2) Using KNIME:



Output:

Rows: 5 | Columns: 2

Table Statistics

☐	#	RowID	Voltage <i>Number (double)</i>	Cluster <i>String</i>
☐	1	Row0	0.335	cluster_0
☐	2	Row1	0.336	cluster_0
☐	3	Row2	0.306	cluster_0
☐	4	Row3	1	cluster_1
☐	5	Row4	0.535	cluster_0

3) Using Weka:

Clusterer output

kMeans

=====

Number of iterations: 2

Within cluster sum of squared errors: 0.06910363630837056

Initial starting points (random):

Cluster 0: 0.9999999

Cluster 1: 0.336347

Missing values globally replaced with mean/mode

Final cluster centroids:

Attribute	Full Data	Cluster#	
		0	1
	(5.0)	(1.0)	(4.0)
=====			
Voltage	0.5026	1	0.3782

Time taken to build model (full training data) : 0 seconds

=== Model and evaluation on training set ===

Clustered Instances

0 1 (20%)

1 4 (80%)



Walchand College Of Engineering, Sangli.

Assignment 11

Aim: Unsupervised Learning Methods : Cluster Analysis - hierarchical based.

Theory:

Hierarchical clustering is a type of cluster analysis that seeks to build a hierarchy of clusters. It has two main forms:

- 1] Agglomerative (Bottom-up) hierarchical clustering:
Each data point starts as its own cluster. Then, pairs of clusters are merged step by step based on similarity, until all belongs to one single cluster. (Using Euclidean distance)
- 2] Divisive (Top-down) hierarchical clustering:
Starts with one large cluster that contains all data points. It is split into smaller clusters based on certain criterion until each data point is its own individual cluster.

Linkage Methods in hierarchical clustering :

1] Single linkage :

Distance between two cluster is defined as minimum distance between any single pair of data points.

2] Complete linkage :

Distance between two cluster is defined as maximum distance between any pair of data points.

3] Average linkage

The distance between two clusters is the average of the distances between all possible pairs of data points.



Numerical :

	Voltage
a	0.3353
b	0.3353
c	0.3065
d	0.9999
e	0.5347

④ Single linkage / Complete linkage / Average linkage

	a	b	c	d	e
a	0				
b	0.0010	0			
c	0.0288	0.0298	0		
d	0.6646	0.6636	0.6934	0	
e	0.1994	0.1984	0.2282	0.4652	0

$$d(x_i, x_j) = |x_i - x_j|$$

Solⁿ using single linkage :

minimum distance = 0.0010

	(a,b)	c	d	e
(a,b)	0			
c	0.0288	0		
d	0.6636	0.6934	0	
e	0.1984	0.2282	0.4652	0

$$\begin{aligned} d(c, (a,b)) &= \min(d(c,a), d(c,b)) \\ &= \min(0.0288, 0.0298) \\ &= 0.0288 \end{aligned}$$

$$\begin{aligned} d(d, (a,b)) &= \min(d(d,a), d(d,b)) \\ &= \min(0.6646, 0.6636) \\ &= 0.6636 \end{aligned}$$

$$\begin{aligned} d(e, (a,b)) &= \min(d(e,a), d(e,b)) \\ &= \min(0.1994, 0.1984) = 0.1984 \end{aligned}$$



Walchand College Of Engineering, Sangli.

minimum distance : 0.0288

	(a,b,c)	d	e
(a,b,c)	0		
d	0.6636	0	
e	0.1984	0.4652	0

$$\begin{aligned}
 d(d, (a,b,c)) &= \min(d(d,ab), d(d,c)) \\
 &= \min(0.6636, 0.6934) \\
 &= 0.6636
 \end{aligned}$$

$$\begin{aligned}
 d(e, (a,b,c)) &= \min(d(e,ab), d(e,c)) \\
 &= \min(0.1984, 0.2282) \\
 &= 0.1984
 \end{aligned}$$

minimum distance : 0.1984

	(a,b,c,e)	d
(a,b,c,e)	0	
d	0.4652	0

$$\begin{aligned}
 d(d, (a,b,c,e)) &= \min(d(d, (a,b,c)), d(d,e)) \\
 &= \min(0.6636, 0.4652) \\
 &= 0.4652
 \end{aligned}$$

$$\begin{aligned}
 \therefore \text{Final cluster} &= (a,b,c,e,d) \\
 &= (0.3353, 0.3363, 0.3065, 0.5347, 0.9999)
 \end{aligned}$$

Solⁿ using Complete linkage : min distance = 0.0010

	(a,b)	c	d	e
(a,b)	0			
c	0.0298	0		
d	0.6646	0.6934	0	
e	0.1994	0.2282	0.4652	0



$$\begin{aligned} d(c, (a,b)) &= \max(d(c,a), d(c,b)) \\ &= \max(0.0288, 0.0298) \\ &= 0.0298 \end{aligned}$$

$$\begin{aligned} d(d, (a,b)) &= \max(d(d,a), d(d,b)) \\ &= \max(0.6646, 0.6636) \\ &= 0.6636 \end{aligned}$$

$$\begin{aligned} d(e, (a,b)) &= \max(d(e,a), d(e,b)) \\ &= \max(0.1994, 0.1984) \\ &= 0.1994 \end{aligned}$$

minimum distance = 0.0298

	(a,b,c)	d	e
(a,b,c)	0		
d	0.6934	0	
e	0.2282	0.4652	0

$$\begin{aligned} d(d, (a,b,c)) &= \max(d(d,ab), d(d,c)) \\ &= \max(0.6646, 0.6934) \\ &= 0.6934 \end{aligned}$$

$$\begin{aligned} d(e, (a,b,c)) &= \max(d(e,ab), d(e,c)) \\ &= \max(0.1994, 0.2282) \\ &= 0.2282 \end{aligned}$$

minimum distance = 0.2282

	(a,b,c,e)	d
(a,b,c,e)	0	
d	0.6934	0

$$\begin{aligned} d(d, (a,b,c,e)) &= \max(d(d,abc), d(d,e)) \\ &= \max(0.6934, 0.4652) \\ &= 0.6934 \end{aligned}$$

final clusters = (0.3353, 0.3363, 0.3065, 0.5347, 0.9999)



Walchand College Of Engineering, Sangli.

Solⁿ using Average linkage:

minimum distance = 0.0010

	ca,b)	c	d	e
ca,b)	0			
c	0.0293	0		
d	0.6641	0.6934	0	
e	0.1989	0.2282	0.4652	0

$$\begin{aligned} d(c, ca,b) &= \text{Avg}(d(c,a), d(c,b)) \\ &= \text{Avg}(0.0288, 0.0293) \\ &= 0.0293 \end{aligned}$$

$$\begin{aligned} d(d, ca,b) &= \text{Avg}(d(d,a), d(d,b)) \\ &= \text{Avg}(0.6646, 0.6636) \\ &= 0.6641 \end{aligned}$$

$$\begin{aligned} d(e, ca,b) &= \text{Avg}(d(e,a), d(e,b)) \\ &= \text{Avg}(0.1994, 0.1984) \end{aligned}$$

minimum distance = 0.0293

	ca,b,c)	d	e
ca,b,c)	0		
d	0.6787	0	
e	0.2135	0.4652	0

$$\begin{aligned} d(d, ca,b,c) &= \text{Avg}(d(d, ca,b), d(d,c)) \\ &= \text{Avg}(0.6641, 0.6934) \\ &= 0.6787 \end{aligned}$$

$$\begin{aligned} d(e, ca,b,c) &= \text{Avg}(d(e, ca,b), d(e,c)) \\ &= \text{Avg}(0.1989, 0.2282) \\ &= 0.2135 \end{aligned}$$



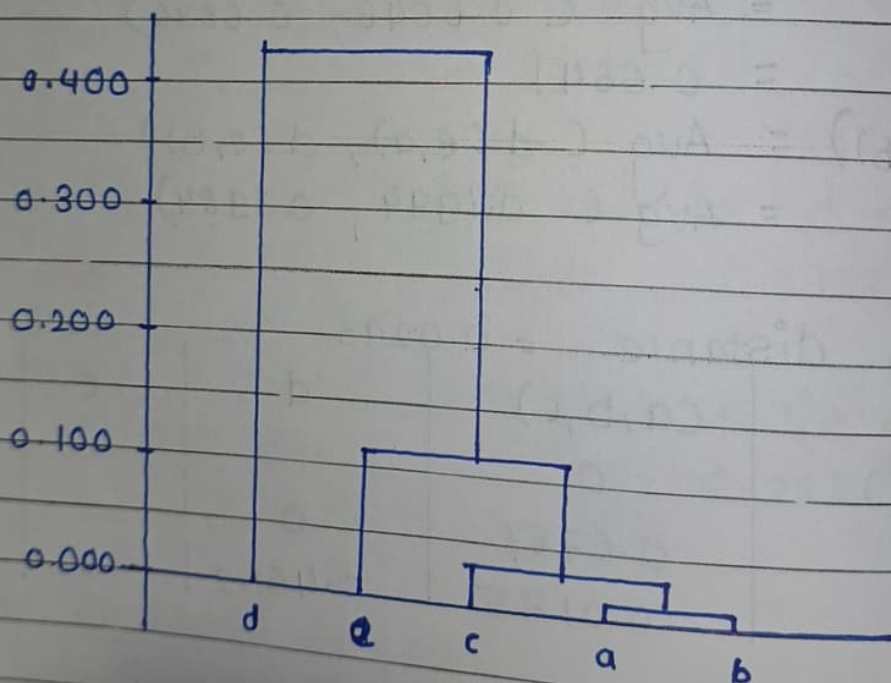
minimum distance = 0.2135

	ca, b, c, e)	d
ca, b, c, e)	0	0
d	0.5719	

$$\begin{aligned}d(d, ca, b, c, e) &= \text{Avg}(d(d, a, b, c), d(d, e)) \\&= \text{Avg}(0.6787, 0.4652) \\&= 0.5719\end{aligned}$$

final cluster : (0.3353, 0.3363, 0.3065, 0.5347, 0.9999)

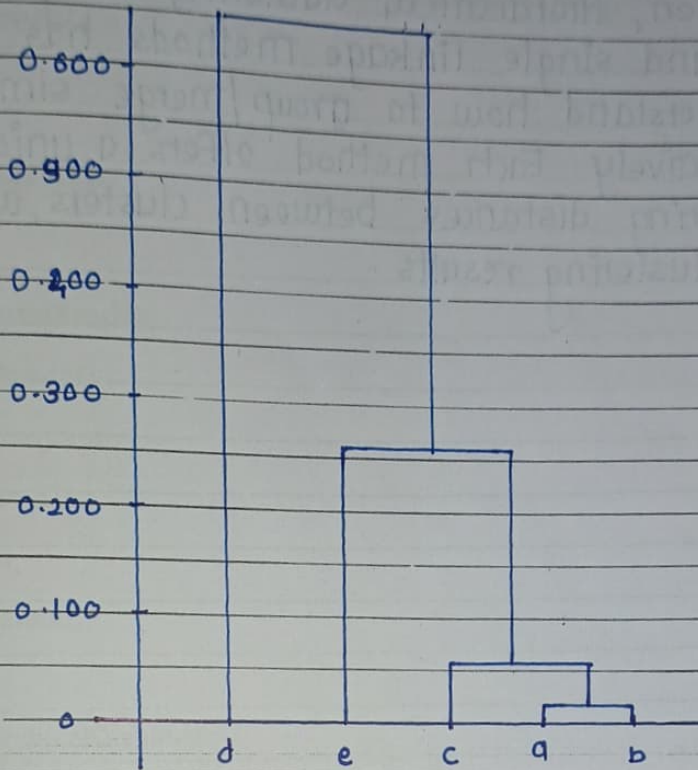
Dendrogram (single linkage) :



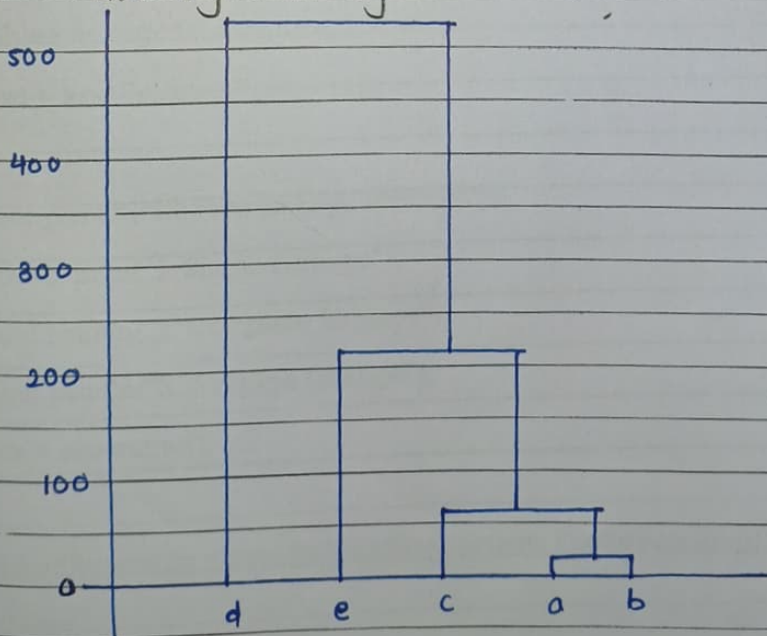


Walchand College Of Engineering, Sangli.

Dendrogram (Complete linkage)



Dendrogram (Average linkage):





Conclusion

In Conclusion, Hierarchical clustering using average, complete and single linkage methods has helped me to understand how to group/merge similar data points effectively. Each method offers a unique approach to measuring distances between clusters, which gives different clustering results.

Assignment 7

Aim: Unsupervised Learning Methods : Cluster Analysis - hierarchical based

1) Using program

Code:

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class HierarchicalClustering {
    public static void main(String[] args) {
        String file = "Mine_Data.csv";
        List<Double> voltages = new ArrayList<>();
        Scanner sc = new Scanner(System.in);

        System.out.println("Choose Linkage Method:");
        System.out.println("1. Single Linkage");
        System.out.println("2. Complete Linkage");
        System.out.println("3. Average Linkage");
        int choice = sc.nextInt();

        try (BufferedReader br = new BufferedReader(new FileReader(file))) {
            String line;
            br.readLine();
            while ((line = br.readLine()) != null) {
```



```
String[] fields = line.split(",");
voltages.add(Double.parseDouble(fields[0]));
}
} catch (IOException e) {
    e.printStackTrace();
}

List<List<Double>> clusters = new ArrayList<>();
for (double v : voltages) {
    List<Double> c = new ArrayList<>();
    c.add(v);
    clusters.add(c);
}

while (clusters.size() > 1) {
    double minDist = Double.MAX_VALUE;
    int idx1 = -1, idx2 = -1;

    for (int i = 0; i < clusters.size(); i++) {
        for (int j = i + 1; j < clusters.size(); j++) {
            double dist = 0;

            switch (choice) {
                case 1:
                    dist = singleLinkage(clusters.get(i), clusters.get(j));
                    break;
                case 2:
                    dist = completeLinkage(clusters.get(i), clusters.get(j));
                    break;
                case 3:
                    dist = averageLinkage(clusters.get(i), clusters.get(j));
                    break;
                default:
```

```
System.out.println("Invalid choice. Exiting.");
return;

}

if (dist < minDist) {
    minDist = dist;
    idx1 = i;
    idx2 = j;
}

}

}

System.out.printf("Merging clusters with minimum distance: %.4f%n", minDist);
clusters.get(idx1).addAll(clusters.get(idx2));
clusters.remove(idx2);
System.out.println("Merged clusters: " + clusters);
}

System.out.println("Final Cluster: " + clusters);
}

private static double singleLinkage(List<Double> c1, List<Double> c2) {
    double minDist = Double.MAX_VALUE;
    for (double v1 : c1) {
        for (double v2 : c2) {
            double dist = Math.abs(v1 - v2);
            if (dist < minDist) {
                minDist = dist;
            }
        }
    }
    return minDist;
}
```

```

private static double completeLinkage(List<Double> c1, List<Double> c2) {
    double maxDist = Double.MIN_VALUE;
    for (double v1 : c1) {
        for (double v2 : c2) {
            double dist = Math.abs(v1 - v2);
            if (dist > maxDist) {
                maxDist = dist;
            }
        }
    }
    return maxDist;
}

```

```

private static double averageLinkage(List<Double> c1, List<Double> c2) {
    double totalDist = 0.0;
    int count = 0;
    for (double v1 : c1) {
        for (double v2 : c2) {
            totalDist += Math.abs(v1 - v2);
            count++;
        }
    }
    return totalDist / count;
}

```

Input (Mine_Data.csv):

Voltage	High	Soil Type	Mine Type
0.335347	0.181818	0.6	1
0.336347	0.272727	0.6	1
0.306495	1	1	3
0.999999	0.181818	0.6	2
0.534743	0.727273	0.6	2

Code Output:

1) Single Linkage:

```
PS C:\Users\khushi\Desktop\DM> javac .\HierarchicalClustering.java
PS C:\Users\khushi\Desktop\DM> java HierarchicalClustering
Choose Linkage Method:
1. Single Linkage
2. Complete Linkage
3. Average Linkage
1
Merging clusters with minimum distance: 0.0010
Merged clusters: [[0.335347054, 0.336347054], [0.306495062], [0.999998728], [0.534742556]]
Merging clusters with minimum distance: 0.0289
Merged clusters: [[0.335347054, 0.336347054, 0.306495062], [0.999998728], [0.534742556]]
Merging clusters with minimum distance: 0.1984
Merged clusters: [[0.335347054, 0.336347054, 0.306495062, 0.534742556], [0.999998728]]
Merging clusters with minimum distance: 0.4653
Merged clusters: [[0.335347054, 0.336347054, 0.306495062, 0.534742556, 0.999998728]]
Final Cluster: [[0.335347054, 0.336347054, 0.306495062, 0.534742556, 0.999998728]]
```

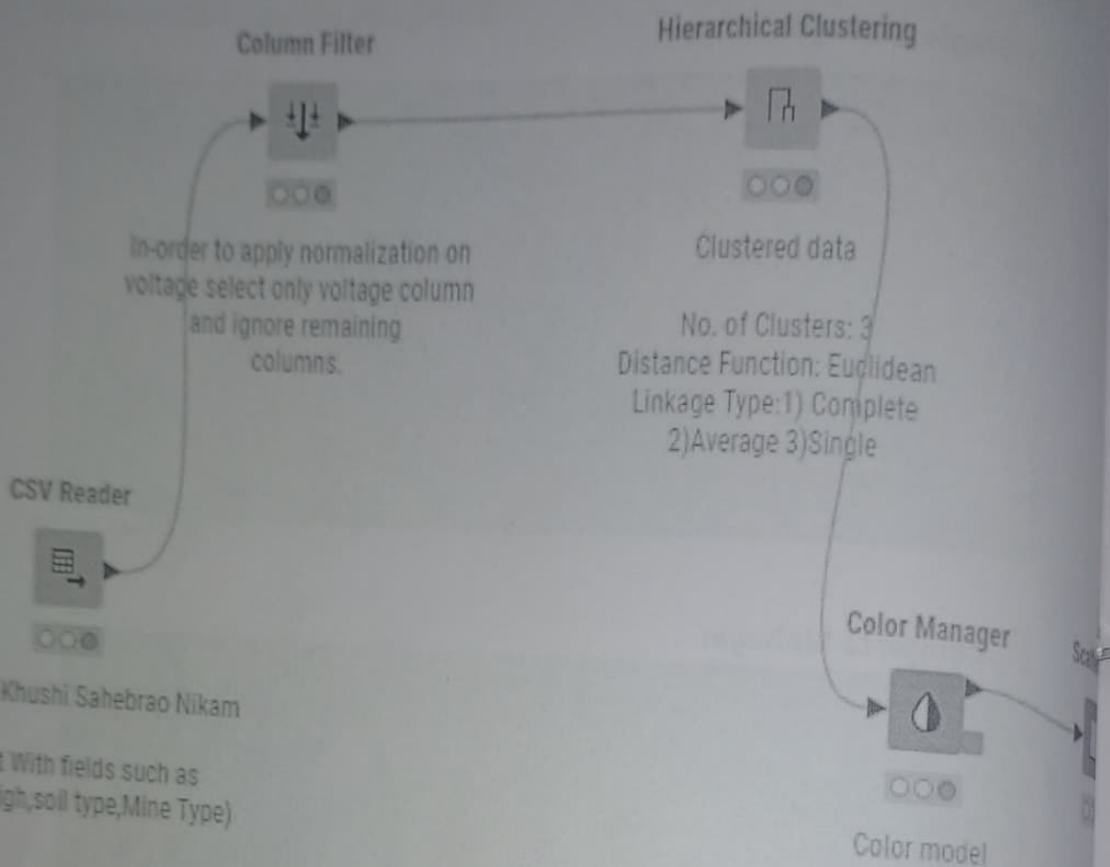
2) Complete Linkage:

```
PS C:\Users\khushi\Desktop\DM> java HierarchicalClustering
Choose Linkage Method:
1. Single Linkage
2. Complete Linkage
3. Average Linkage
2
Merging clusters with minimum distance: 0.0010
Merged clusters: [[0.335347054, 0.336347054], [0.306495062], [0.999998728], [0.534742556]]
Merging clusters with minimum distance: 0.0299
Merged clusters: [[0.335347054, 0.336347054, 0.306495062], [0.999998728], [0.534742556]]
Merging clusters with minimum distance: 0.2282
Merged clusters: [[0.335347054, 0.336347054, 0.306495062, 0.534742556], [0.999998728]]
Merging clusters with minimum distance: 0.6935
Merged clusters: [[0.335347054, 0.336347054, 0.306495062, 0.534742556, 0.999998728]]
Final Cluster: [[0.335347054, 0.336347054, 0.306495062, 0.534742556, 0.999998728]]
```

3) Average Linkage:

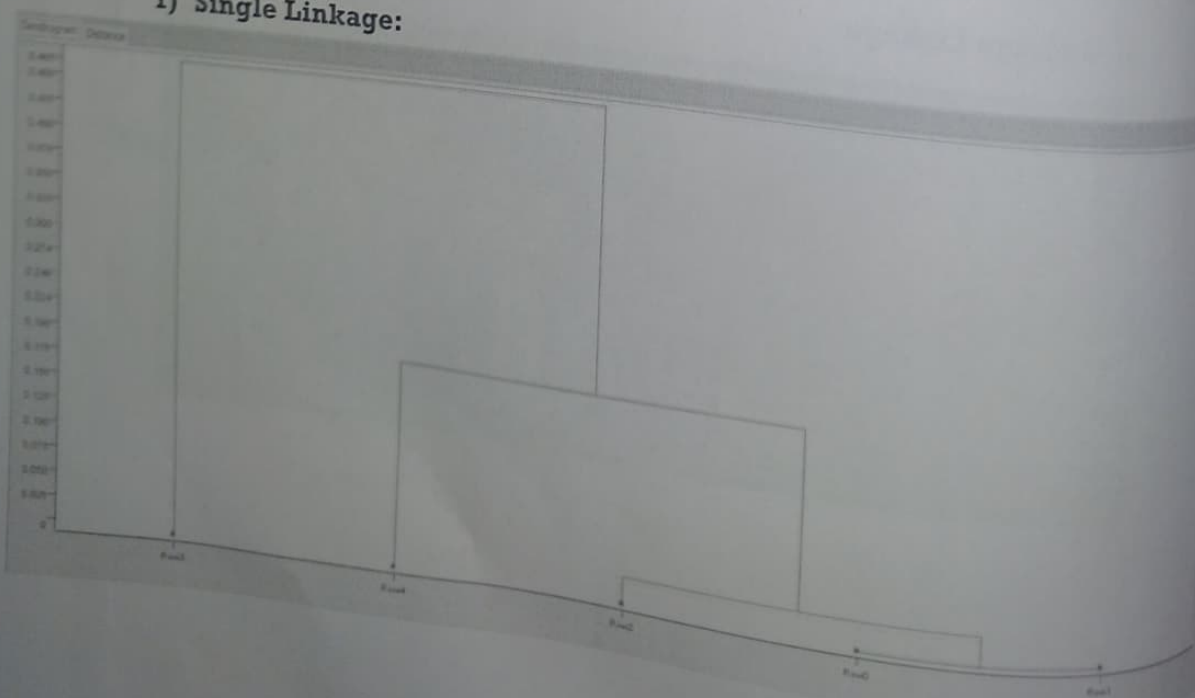
```
PS C:\Users\khushi\Desktop\DM> java HierarchicalClustering
Choose Linkage Method:
1. Single Linkage
2. Complete Linkage
3. Average Linkage
3
Merging clusters with minimum distance: 0.0010
Merged clusters: [[0.335347054, 0.336347054], [0.306495062], [0.999998728], [0.534742556]]
Merging clusters with minimum distance: 0.0294
Merged clusters: [[0.335347054, 0.336347054, 0.306495062], [0.999998728], [0.534742556]]
Merging clusters with minimum distance: 0.2087
Merged clusters: [[0.335347054, 0.336347054, 0.306495062, 0.534742556], [0.999998728]]
Merging clusters with minimum distance: 0.0218
Merged clusters: [[0.335347054, 0.336347054, 0.306495062, 0.534742556, 0.999998728]]
Final Cluster: [[0.335347054, 0.336347054, 0.306495062, 0.534742556, 0.999998728]]
```

2) Using KNIME:

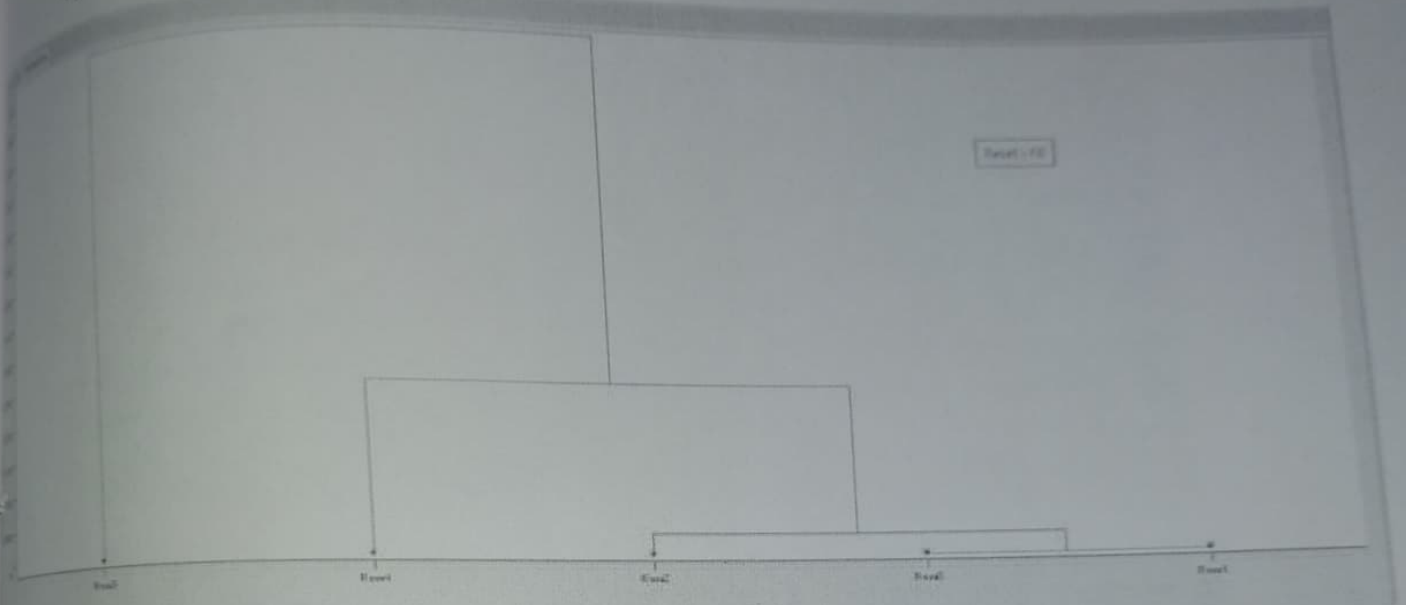


Output:

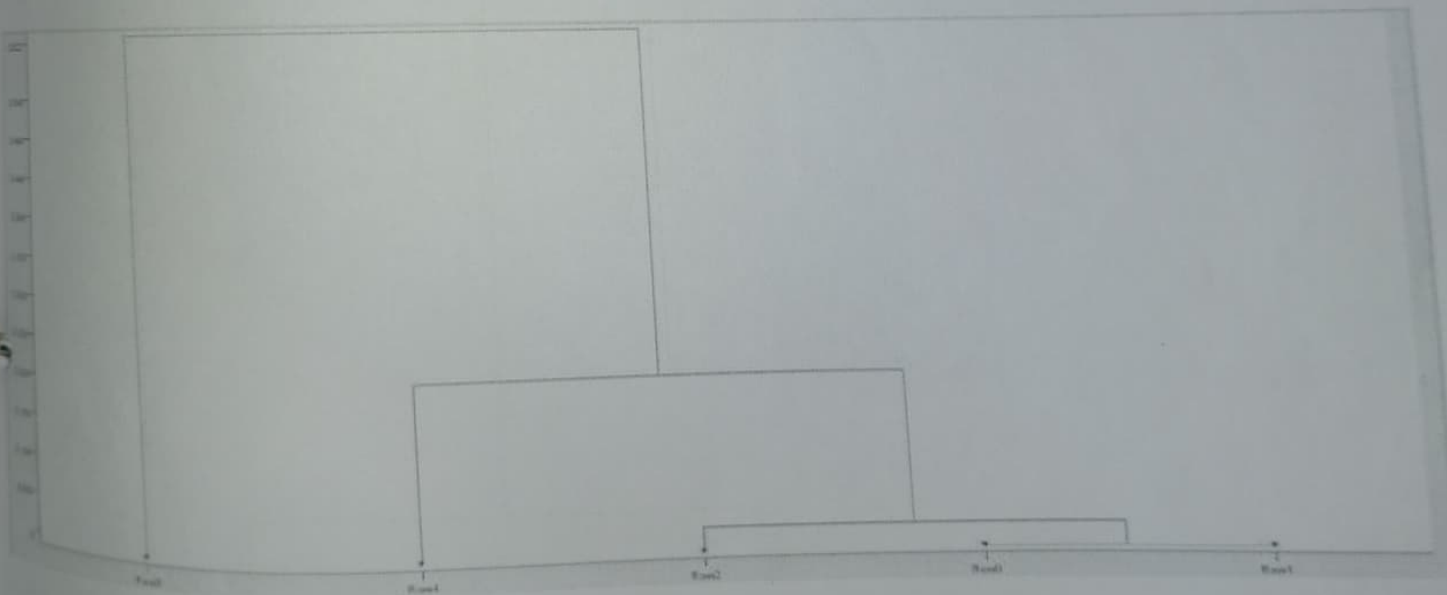
1) Single Linkage:



2) Complete Linkage:



3) Average Linkage



3) Using Weka:

1) Single Linkage:

```

Cluster output
--- Run information ---
Scheme: weka.clusterers.HierarchicalClusterer -N 2 -L SINGLE -P -A "weka.core.EuclideanDistance" -B first-last
Instances: Mine_Data
Instances: 1
Attributes: 1
Test mode: evaluate on training data

--- Clustering model (full training set) ---
Cluster 0
1(0.335347054;0.50144,0.336347054;0.00144);0.04016,0.306495062;0.0416;0.24447,0.534742556;0.28605)

Time taken to build model (full training data) : 0 seconds
--- Model and evaluation on training set ---
Clustered Instances
0      4 ( 50%)
1      1 ( 12%)
    
```

Weka Cluster Tree Visualizer: 18:50:47 - HierarchicalClusterer (Mine_Data)



2) Complete Linkage:

```
Cluster output
*** Run information ***
Scheme: weka.clusterers.HierarchicalClusterer -N 2 -L COMPLETE -P -A "weka.core.EuclideanDistance -R first-last"
Instance: Mine_Data
Instances: 5
Attributes: 1
Test mode: evaluate on training data
```

```
*** Clustering model (full training set) ***
```

```
Cluster 0
(0.335347054;0.00144,0.336347054;0.00144):0.0416,0.306495062;0.04305):0.28608,0.534742556;0.32912)
```

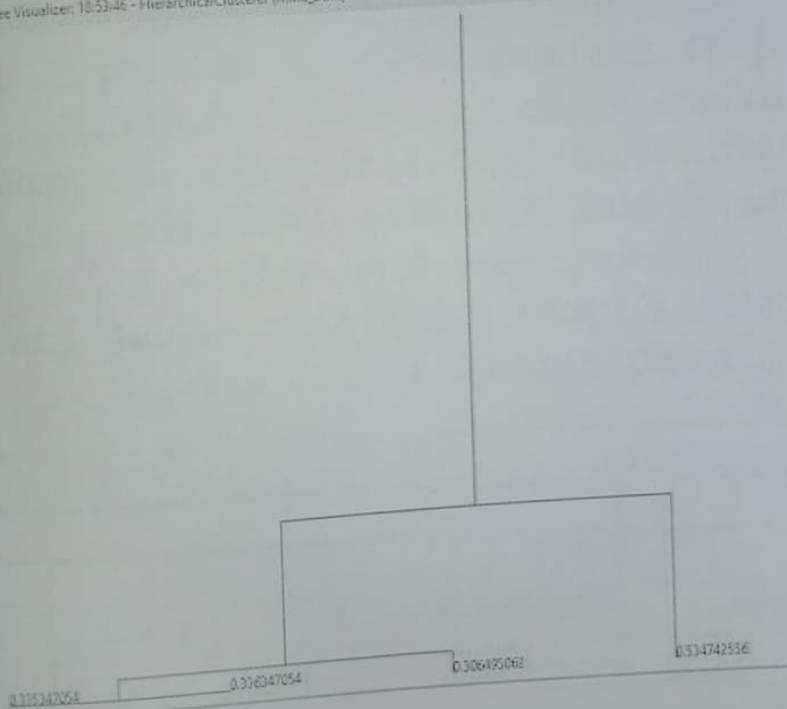
```
Time taken to build model (full training data) : 0 seconds
```

```
*** Model and evaluation on training set ***
```

```
Clustered Instances
```

```
0 4 ( 80%)
1 1 ( 20%)
```

Weka Cluster Tree Visualizer: 18:53:46 - HierarchicalClusterer (Mine_Data)



3) Average Linkage:

Clusterer output

=== Run information ===

Scheme: weka.clusterers.HierarchicalClusterer -N 2 -L AVERAGE -P -A "weka.core.EuclideanDistance -R first-last"
 Relation: Mine_Data
 Instances: 5
 Attributes: 1
 Voltage
 Test mode: evaluate on training data

=== Clustering model (full training set) ===

Cluster 0

{{(0.335347054;0.00144,0.336347054;0.00144):0.04088,0.306495062;0.04232):0.25958,0.534742556;0.30091}}

Time taken to build model (full training data) : 0 seconds

=== Model and evaluation on training set ===

Clustered Instances

0 4 (80%)
 1 1 (20%)

Weka Cluster Tree Visualizer: 10:55:06 - HierarchicalClusterer (Mine_Data)

